Chapter 2

Embedded System Hardware

Topics

2.1 LCD Screen and Touchscreen Panel

2.2 Digital Output and Input

2.3 Analog Output and Input

2.4 Clock and Timers

2.5 Serial Communications Interfaces

Topics

2.1 LCD Screen and Touchscreen Panel

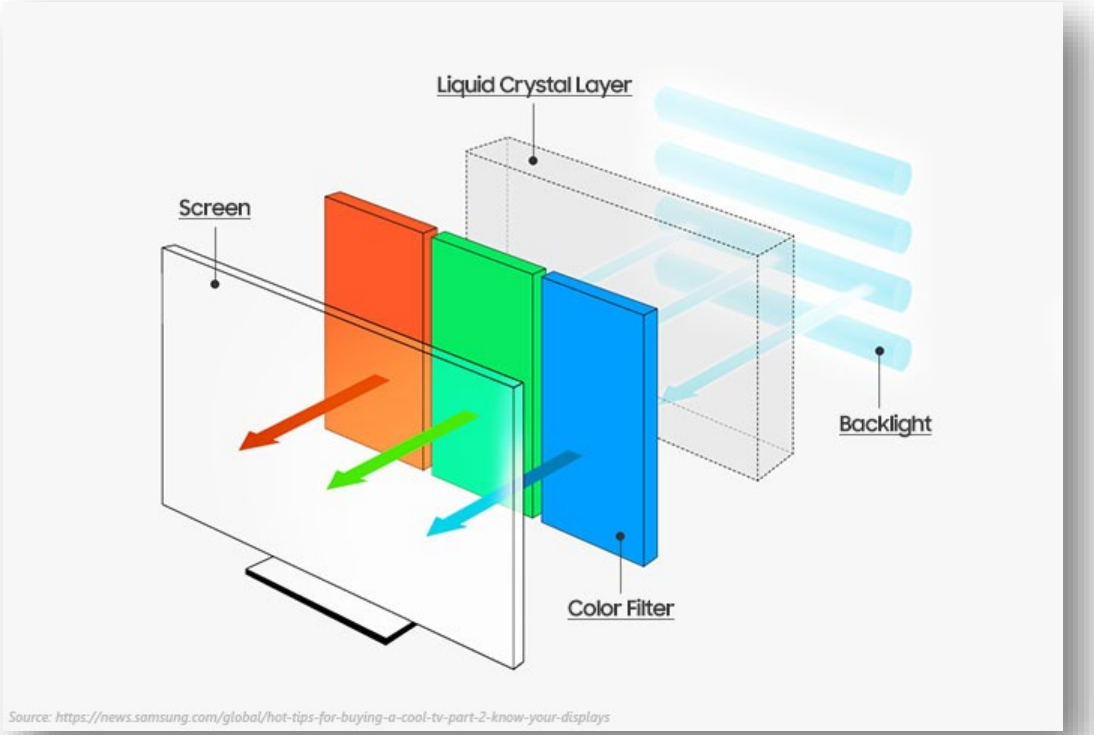
- Liquid Crystal Display (LCD) Screen
  - Types of LCD Screen
  - Graphic LCD Screen Pixels and Resolution
  - Graphics System Hardware Components
  - Graphics Software Library
- Touchscreen Panel
  - Types of Touchscreen Panel
  - Touch System Hardware Components

Liquid Crystal Display (LCD) Screen

Liquid Crystal Display (LCD) Screen

- A Liquid Crystal Display (LCD) screen is a **flat-panel display** that uses the **light-modulating properties of liquid crystals**.
- Liquid crystals **do not emit light directly**, instead **using a backlight or reflector** to produce **arbitrary or fix images** in color or monochrome.
- LCD screens are **used in a wide range of embedded devices** including digital cameras, watches, calculators, and Blu-ray players.

Liquid Crystal Display (LCD) Screen



Types of LCD Screen

- LCD screens can be broadly classified into 3 types:
  - Segment LCD
  - Dot Matrix LCD
  - **Graphic LCD**

Segment LCD

Dot Matrix LCD

Graphic LCD

Graphic LCD Screen Pixels and Resolution

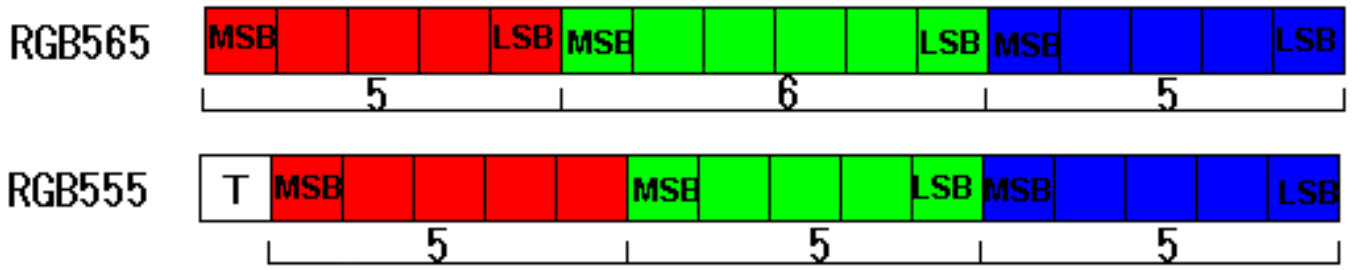
- A Graphic LCD screen is made up of **pixels**. Every pixel can show one point of color and each pixel is composed of three color points: **Red, Green and Blue**.

- Each of these three basic colors (RGB) can be **represented by 8-bits**. Therefore, with 24-bits, **16 million colors (2^{24})** can be produced. This is termed as "True Color"

Graphic LCD Screen Pixels and Resolution

- It is also common to use **16-bits to represent these three RGB colors**. With 16-bits, **64K colors (2^{16})** can be produced. This is termed as "High Color"

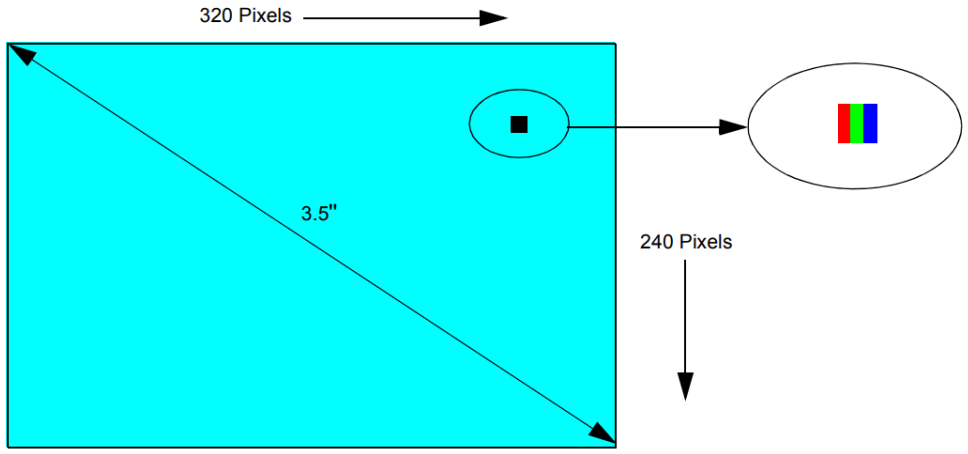
- To divide 16-bits among Red, Green and Blue color, two schemes are used:
  - Scheme 1: **(R<5>, G<6>, B<5>)**
  - Scheme 2: **(T<1>, R<5>, G<5>, B<5>)**



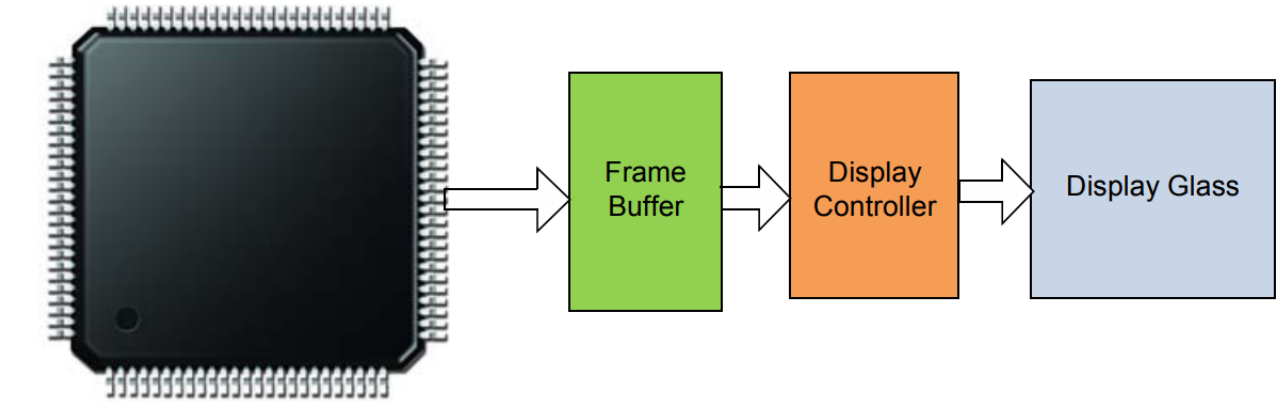
Graphic LCD Screen Pixels and Resolution

- The number of such pixels in horizontal and vertical directions is called the **screen resolution**. For example, a resolution of **320x240** means there are **320 pixels horizontally** (number of columns) and **240 pixels vertically** (number of rows).

- The diagonal length of the display screen is termed as the length of the display and is usually measured in unit **inch (")**.



Graphics System Hardware Components



Source: http://www.microchip.com

Graphics System Hardware Components

- There are four basic hardware components for any embedded graphics system. They consist of the **display glass**, **display controller**, **frame buffer** and the **microcontroller**.

- **Display glass** is the device which **displays a sequence of colors on the pixels** and also converts the digital representation of colors to actual colors. The display glass has no inherent memory and **must be constantly updated with pixel data** in each row and column, failing which, the display will go blank.

- **Display controller fetches data** from the frame buffer, **decodes it** to the required bit format and **feeds it** to the display glass along with proper control signals.

Graphics System Hardware Components

- **Frame buffer** is a **memory** usually a RAM, which **holds the data to be shown** on the display screen and acts as the data source for the display controller. The required **size of the frame buffer** depends on the **resolution and color depth**.

Frame Buffer Size (Bytes) = Number of Pixels x Color Depth (Bits) / 8

- **Microcontroller** allows the **application code running inside** to **decide which data should be stored in the frame buffer**, and as the frame buffer changes, the display content also changes.

Graphics System Hardware Components

Example:

Calculate the Frame Buffer size needed for a QVGA (320x240) display using a 16 BPP (bit per pixel) color depth.

Answer: **Frame Buffer** **= 320 x 240 x 16/8**

= 76800 x 16/8

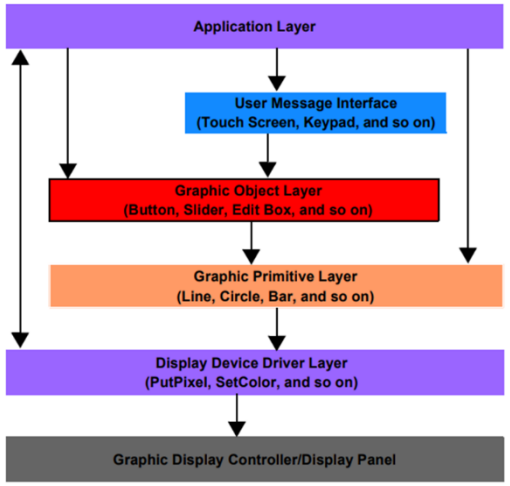
= 153600 bytes

Graphics Software Library

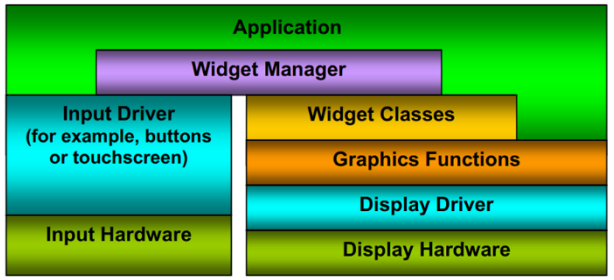
- Graphics Software Library provides a **collection of graphics functions** that allowing the development of **compelling user interfaces** on the LCD screen attached to the microcontrollers.

- The **structure of the Graphics Software Library** developed by different vendors are **usually different**. However, the **basic library component** required for any graphics application is the **Display Driver** which provides one basic operation (i.e., setting the color of a pixel).

Graphics Software Library



Microchip Graphic Library Structure



TivaWare Graphic Library Structure

It’s  time!

Question 1

Which of the following LCD screens can be used to produce rich color images?

Answer:

A. Segment LCD screen

B. Dot Matric LCD screen

C. Graphic LCD screen

Question 2

Calculate the total number of pixels of a LCD screen with resolution of 240x320.

Answer:

A. 76800 pixels

B. 67800 pixels

C. 87600 pixels

Question 3

Why is frame buffer necessary in the LCD Graphics Hardware System?

Answer:

A. To displays a sequence of colors on the pixels.

B. To holds the data to be shown on the display.

C. To allows the application code to run.

Touchscreen Panel

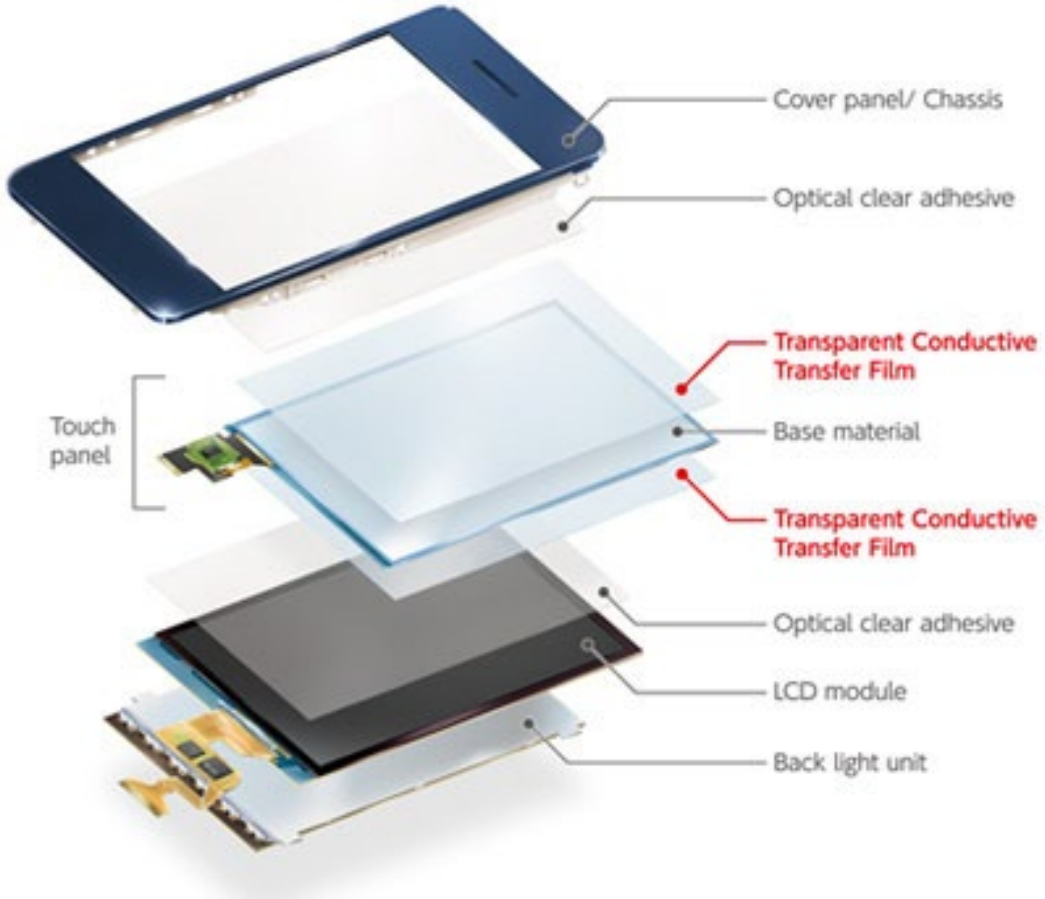
Touchscreen Panel

- A Touchscreen panel is a **thin transparent panel** usually layered **on top of a LCD screen** to **allow the user to interact** with the embedded device **by touching** directly on the LCD screen.

- The **touch** can be of a **human finger, hand, pointed finger nail or passive objects like stylus**. Users can simply move things on the screen, scroll them, zoom in or out and many more.

- Touchscreens panel are commonly used in embedded devices such as **game consoles, tickets vending machines, banks ATM, digital photo printers and etc**.

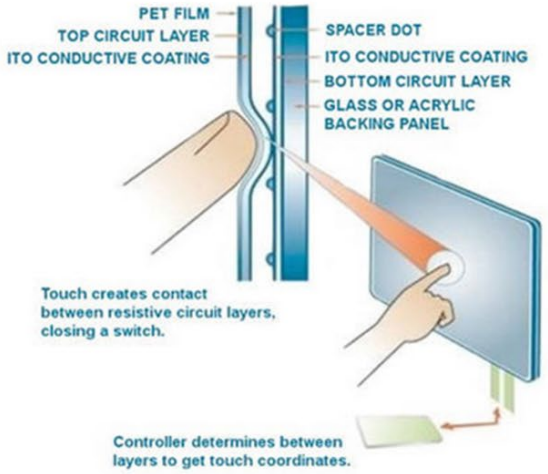
Touchscreen Panel



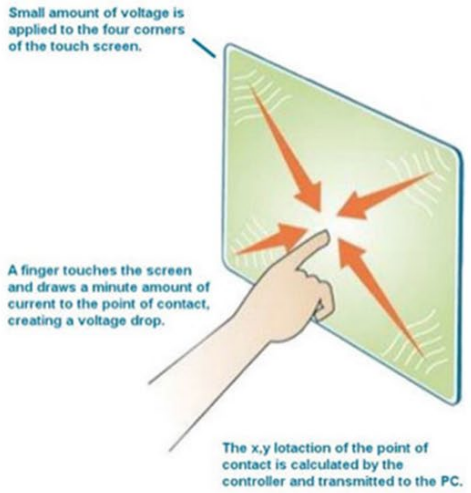
Source: http://www.hitachi-chem.co.jp

Types of Touchscreen Panel

- Touchscreen panel differentiate with different sensing technologies:
  - **Resistive**
  - **Capacitive**
  - Surface Acoustic Wave
  - Infrared



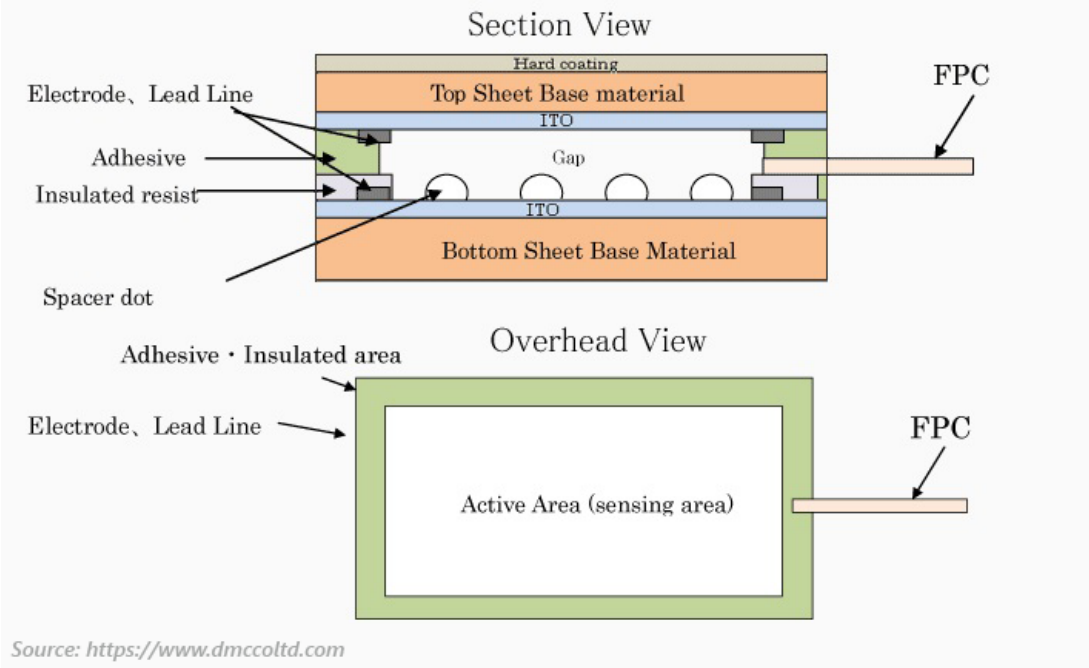
Resistive Touchscreen



Capacitive Touchscreen

Types of Touchscreen Panel

- Resistive Technology – Analog 4-wire Structure

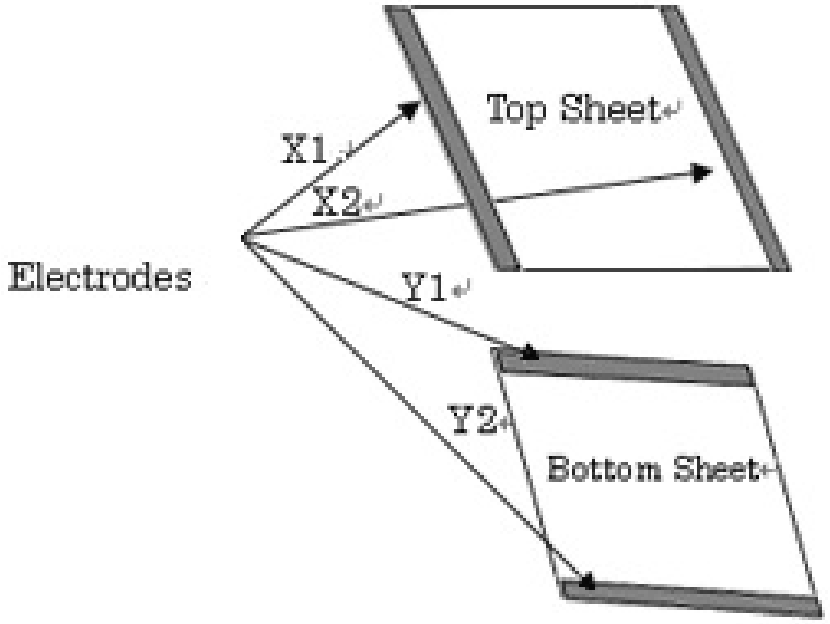


Types of Touchscreen Panel

- Resistive Technology – Analog 4-wire Structure
  - It consists of **top and bottom transparent sheets coated with ITO** (Indium Tin Oxide) facing each other **with a gap between them**. ITO is a transparent conducting material.
  - **Spacer dots** are usually printed on the **bottom sheets to prevent the top and bottom sheets from contacting** when not pressed.
  - **Electrodes** are placed on **edges** of the transparent sheets to perform resistive sensing by measuring voltages.
  - These electrodes are going into **FPC via lead lines** that are also placed on edges, and connected with external connector.

Types of Touchscreen Panel

- Resistive Technology – Analog 4-wire Sensing



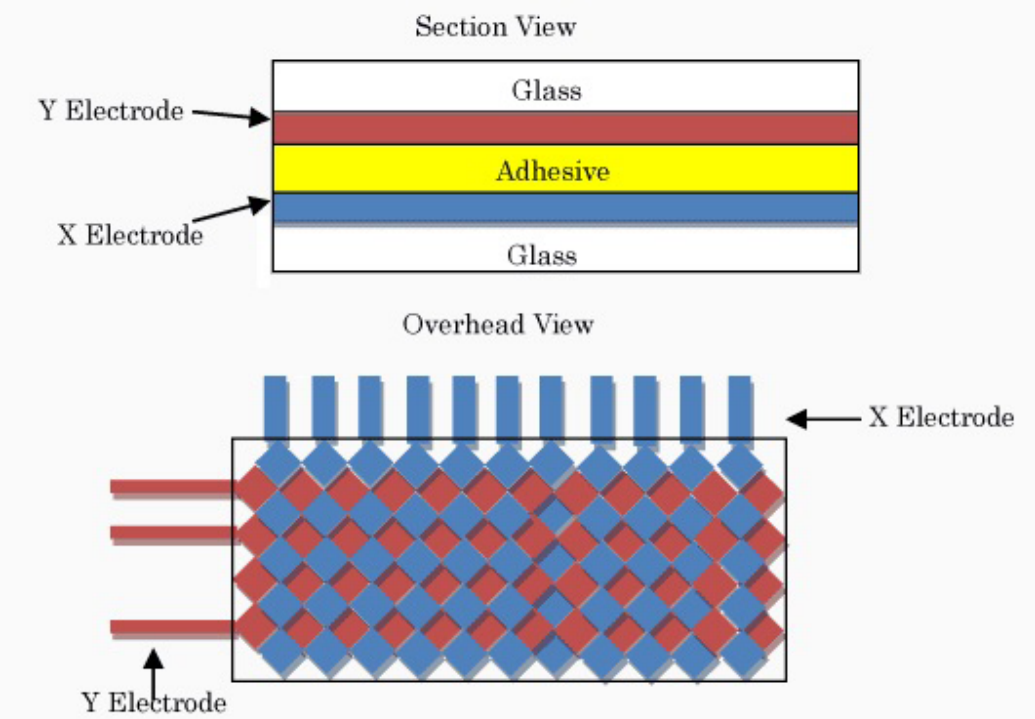
Source: https://www.dmccoltd.com

Types of Touchscreen Panel

- Resistive Technology – Analog 4-wire Sensing
  - **Top sheet** has **electrodes (X)** for the **vertical direction** while the **bottom sheet** has **electrodes (Y)** for the **horizontal direction**.
  - **Voltage is imposed** between the **electrode X1 (0v)** and **X2 (5v)** on the top sheet. If the **central point between the X1 and X2 is touched**, **2.5V would be measured** by the **electrode (Y)** at the bottom sheet.
  - After detecting the coordinate point in X direction, voltage is imposed between the **electrode Y1 (0v) and Y2 (5v)** on the bottom sheet. The **point between Y1 and Y2 would be measured** by **electrode (X)** at the top sheet.
  - In this way, the touched point in X and Y directions is detected and measured.

Types of Touchscreen Panel

- Capacitive Technology – Projected Capacitive Structure



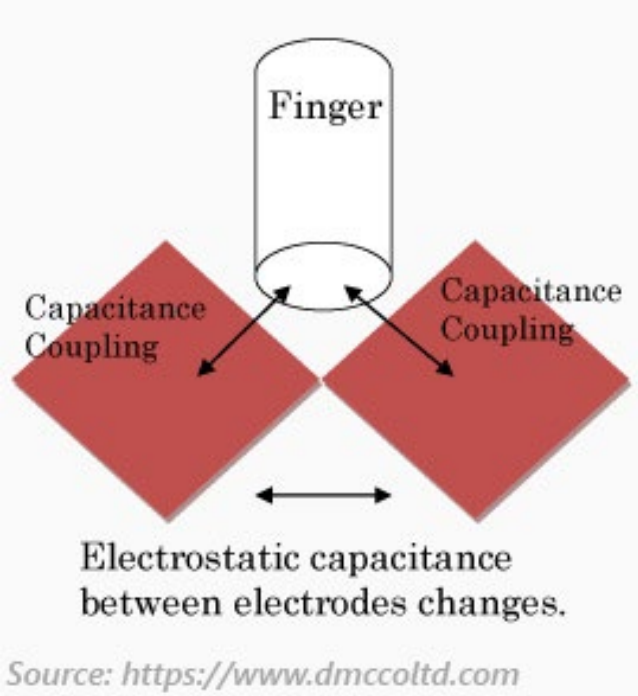
Source: https://www.dmccoltd.com

Types of Touchscreen Panel

- Capacitive Technology – Projected Capacitive Structure
  - **X electrodes** are forming on one glass, and **Y electrodes** are forming on another glass.
  - The **two glass sheets** are laminated in the way that **two electrode sides are facing**.
  - The **X and Y** electrodes are **intersecting in matrix**.

Types of Touchscreen Panel

- Capacitive Technology – Projected Capacitive Sensing



Types of Touchscreen Panel

- Capacitive Technology – Projected Capacitive Sensing
  - When a finger comes close to the patterning of X and Y electrodes, **a capacitance coupling will occur** between the **finger and the electrodes** and makes the electrostatic capacitance between the X and Y electrodes change.
  - The touch sensor **detects touched points** as it **constantly checks** on the electrode lines where the electrostatic capacitance has changed.

Touch System Hardware Components

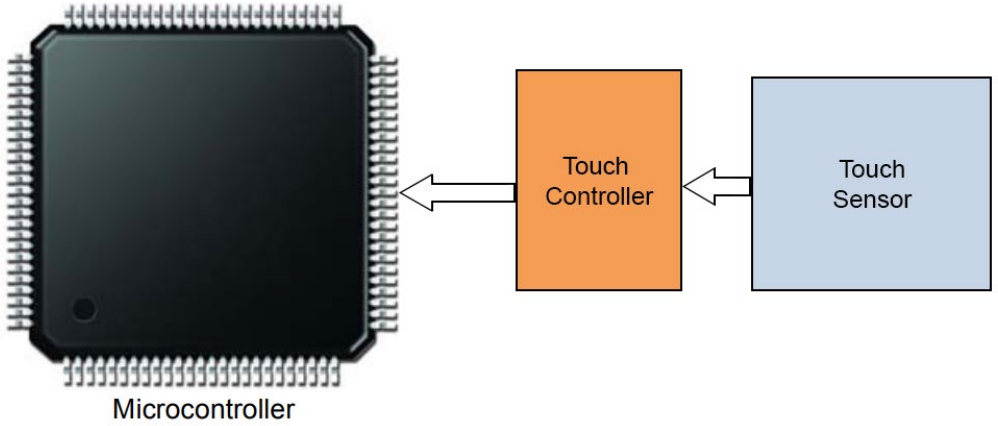
- There are three basic hardware components for any embedded touchscreen system. They consist of the **touch sensor**, **touch controller** and the **microcontroller**.

- **Touch sensor** is a clear glass panel with a **touch responsive surface**. The sensor has an electrical current or signal going through it and **touching the screen** can **cause a voltage or signal change**. This change is used to determine the location of the touch to the screen.

- **Touch controller** connects between the touch sensor and microcontroller. It **takes information from the touch sensor** and **translate** it into information that microcontroller can understand.

Touch System Hardware Components

- **Microcontroller** allows the **touchscreen software driver** code running inside to know how to interpret the touch event information that is sent from the touch controller.



It’s  time!

Question 1

Which of the following touchscreen panel can support multi-touch?

Answer: A. Resistive Touchscreen Panel

B. Capacitive Touchscreen Panel

Question 2

Which of the following touchscreen panel uses less electrode to perform sensing on the touch location?

Answer: A. Resistive Touchscreen Panel

B. Capacitive Touchscreen Panel

Question 3

Which of the following is NOT part of the touch system hardware components?

Answer:

A. Touch controller

B. Touch sensor

C. Display controller

Topics

2.2 Digital Output and Input

- Digital Output
  - Digital Output Type
- Digital Input
  - Digital Input Type

Digital Output

Digital Output

- Digital output in the microcontroller provides a means to **generate** a **high** or a **low** logic level.
- When a pin is configured at **digital output high** logic level, the output **voltage** measured at the pin is typically **close to V_{DD}**.
- When a pin is configured at **digital output low** logic level, the output **voltage** measured at the pin is typically **close to 0 V**.
- Digital output is very often used for **controlling LEDs** or **other high current components** through transistors or relays.

Digital Output

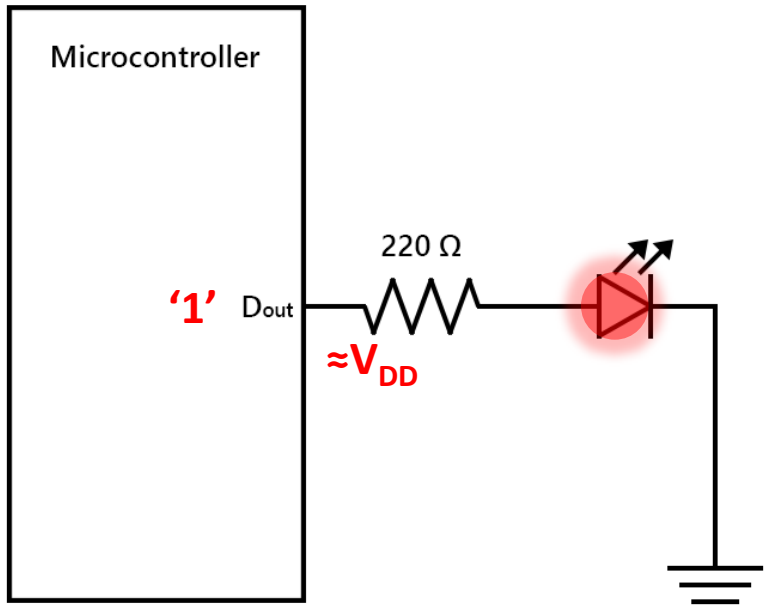
Example 1

What is the **voltage** required at pin D_{out} to light up the LED?

Answer: **$\approx V_{DD}$**

Hence, what is the **logic level** required at pin D_{out} to light up the LED?

Answer: **High Logic Level**



Digital Output

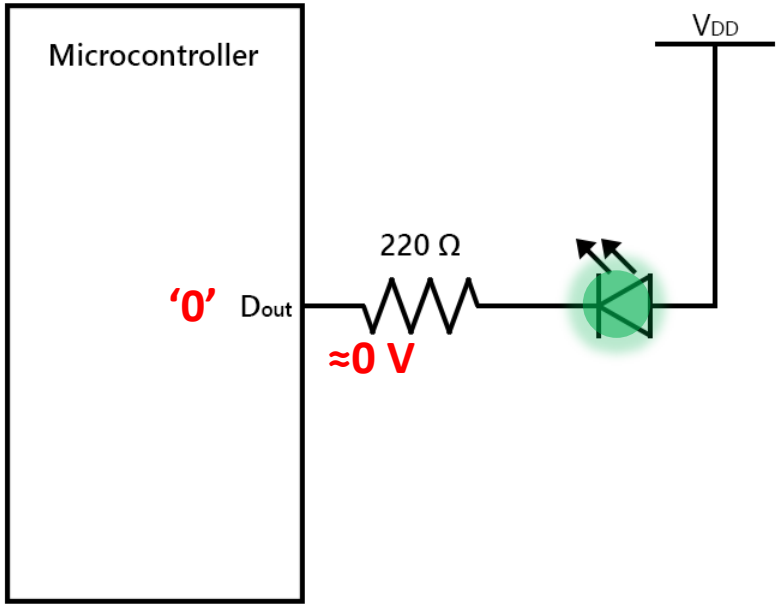
Example 2

What is the **voltage** required at pin D_{out} to light up the LED?

Answer: **≈0 V**

Hence, what is the **logic level** required at pin D_{out} to light up the LED?

Answer: **Low Logic Level**



Digital Output

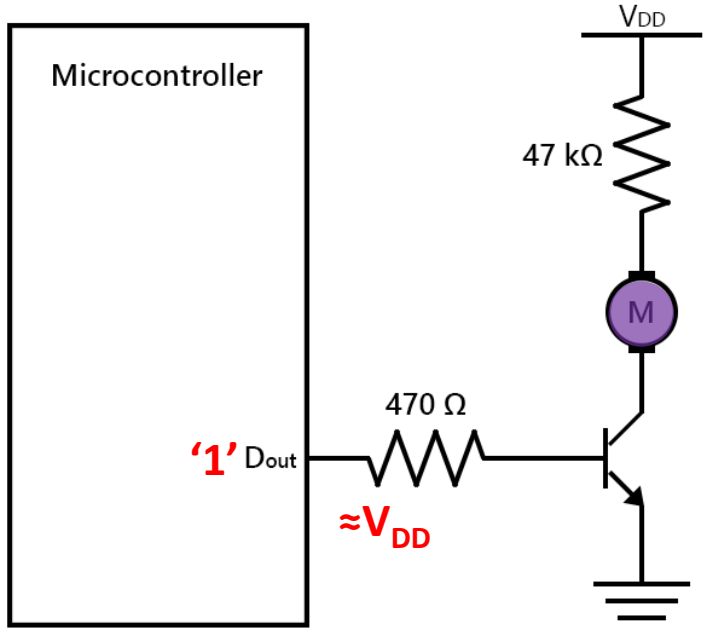
Example 3

What is the **voltage** required at pin D_{out} to activate the vibrator?

Answer: **≈V_{DD}**

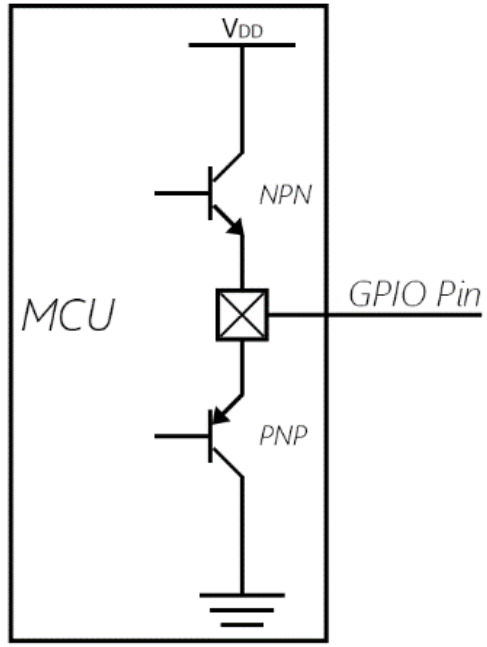
Hence, what is the **logic level** required at pin D_{out} to activate the vibrator?

Answer: **High Logic Level**



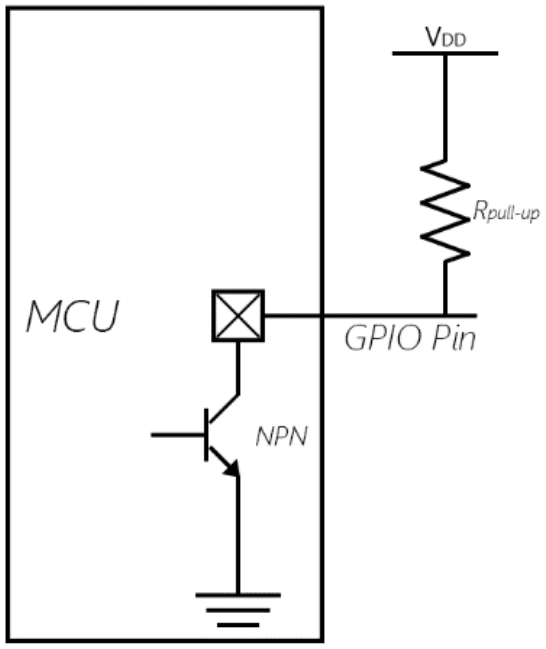
Push-pull Output

- A **Push-pull output** is a digital output that is able to **sink** and **source current** depending on the output logic level.
- When the **output logic level goes low**, it **sinks current** to the internal ground through an internal transistor.
- When the **output logic level goes high**, it **sources current** from the internal power supply through a internal transistor.



Open-collector / drain Output

- An **Open-collector / drain output** is a digital output that **can only sink current**.
- When the **output logic level goes low**, it **sinks current** to the internal ground through an internal transistor.
- When the **output logic level goes high**, it is open with **high impedance**.
- It is usually operate **with an external pull-up resistor** to hold the output logic level high if not driven to low.



Digital Output Type Comparison

- Push-pull Output VS Open-collector / drain Output

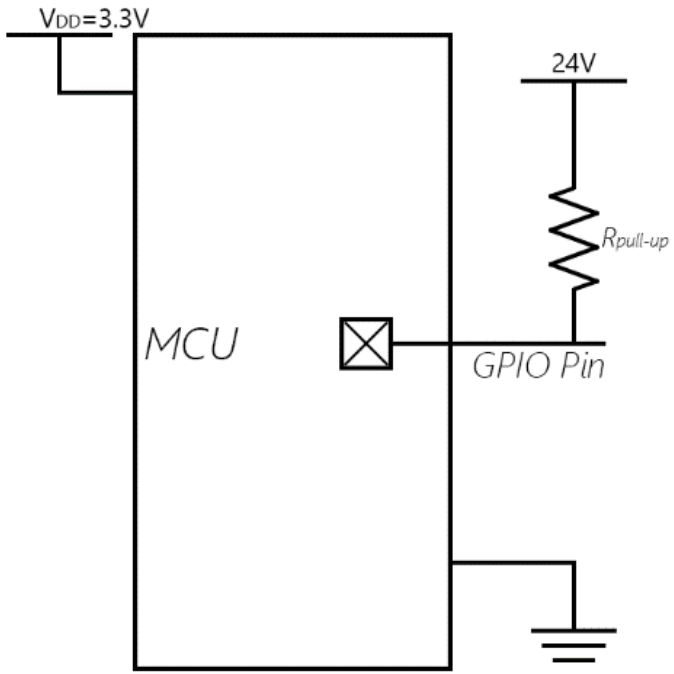
	Push Pull	Open-collector/drain
Switching Time	**Fast** as output is driven both way by transistors	**Slow** as it can on be as fast as the RC time constant allows due to the pull-up resistor.
Connection	**Cannot be connected directly**. If one device pushes and the other pulls, the transistors can be damaged.	**Can be connected directly**. Any of the devices can pull the common line to ground without damaging the transistors.
Pull-up resistor	**Not required**	**Required** and can be pulled-up to any voltage found in the system.

Digital Output

Example 1

- Which digital output type is suitable for the circuit?

- Answer: **Open-collector / drain**



It’s  time!

Question 1

Comparing both GPIO output types, which output type has faster switching time?

Answer: A. Push-pull

B. Open-collector / drain

Question 2

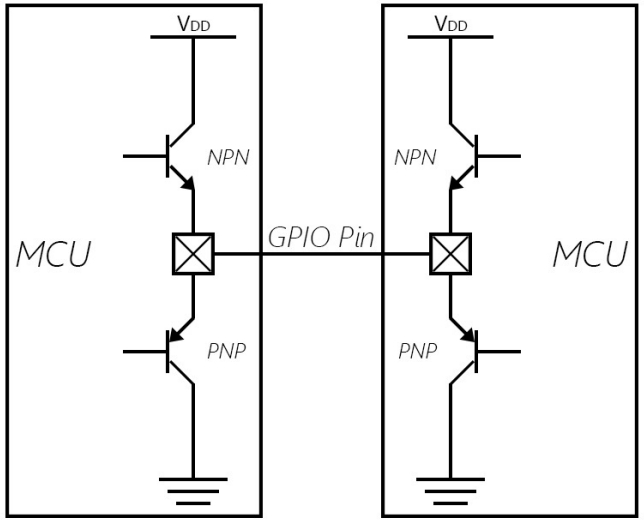
What will happen when two GPIO with push-pull output type connected together having different logic level?

Answer:

A. Nothing will happen.

B. Both transistors will be damaged.

C. It will create a 0 logic level.



Digital Input

Digital Input

- Digital input in a microcontroller is used to **sense true of false activity** in the physical world such as **door opened** or **door closed**.

- It is through an **applied electrical signal** to determine whether the **circuit** is **opened** or **closed**. **Current flows** if the circuit is closed, registering a "**1**" in the digital input. Conversely, an open circuit with **no current flow**, registering a "**0**" at the digital input.

- The most common type of digital input is the **contact closure**. Essentially a **sensor** or **switch closes** or **opens** a set of contacts in accordance with some process change.

Digital Input

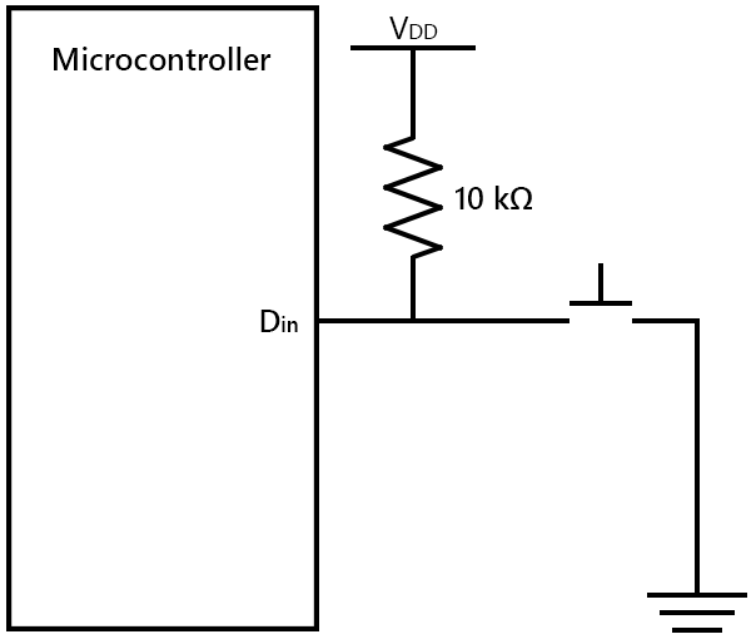
Example 1

What is the **voltage** at D_{in} when the push button is **opened**?

Answer: **≈V_{DD}**

Hence, what is the **logic level** registered at D_{in} when the push button is **opened**?

Answer: **1**



Digital Input

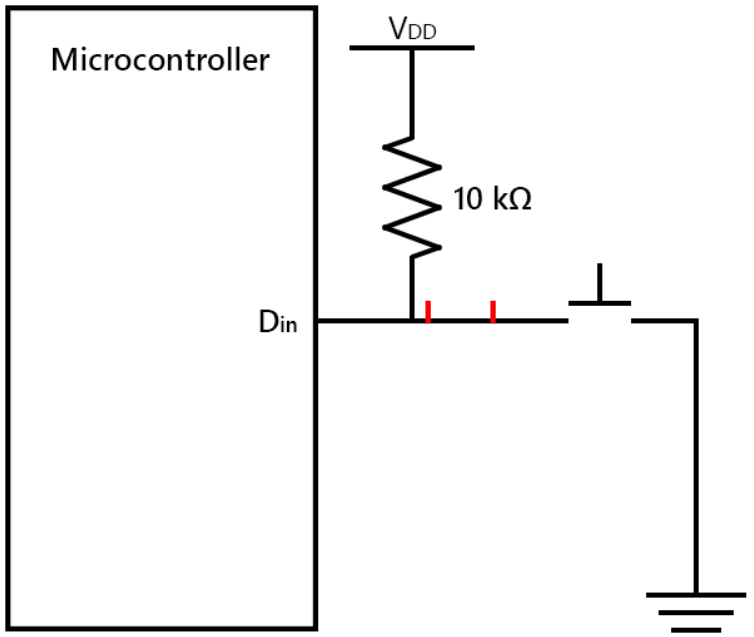
Example 2

What is the **voltage** at D_{in} when the push button is **closed**?

Answer: **≈ 0 V**

Hence, what is the **logic level** registered at D_{in} when the push button is **closed**?

Answer: **0**



Digital Input

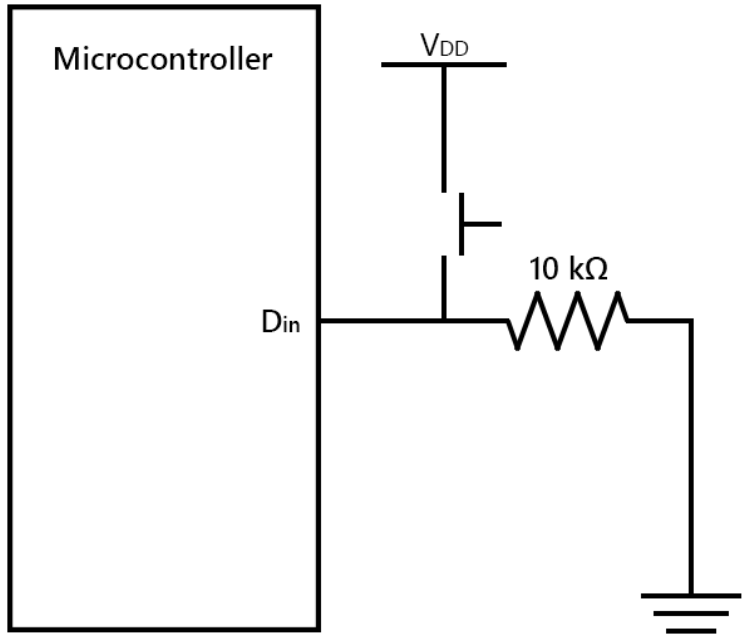
Example 3

What is the **logic level** registered at pin D_{in} when the push button is **opened**?

Answer: **0**

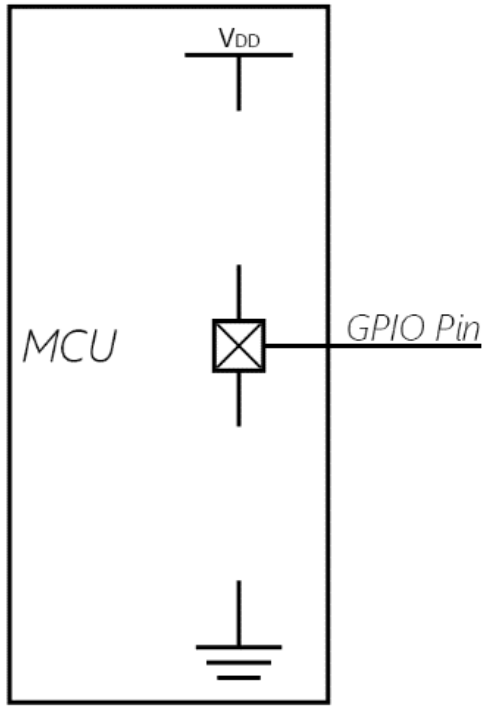
What is the **logic level** registered at pin D_{in} when the push button is **closed**?

Answer: **1**



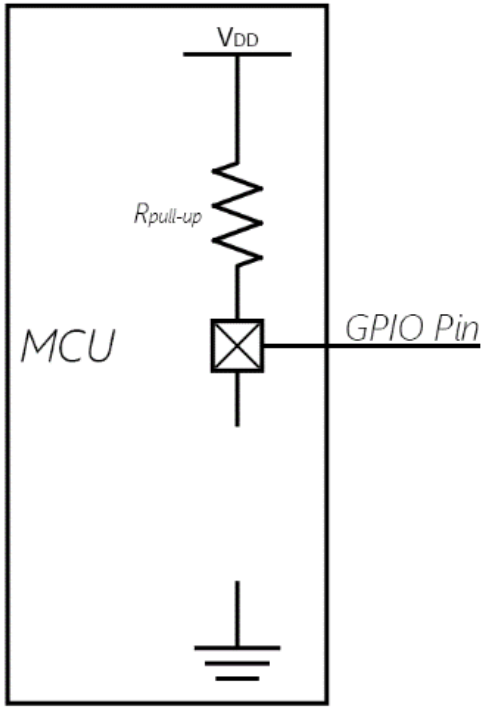
Floating or High-Impedance

- A **Floating** or **High-impedance** or **Tri-stated** input is a digital input that **is left unconnected.**
- It is **neither** in a **high nor low** logic level.
- The unknown input logic level is **not recommended** as the input logic level oscillation may cause more power to be wasted as heat in the transistor.



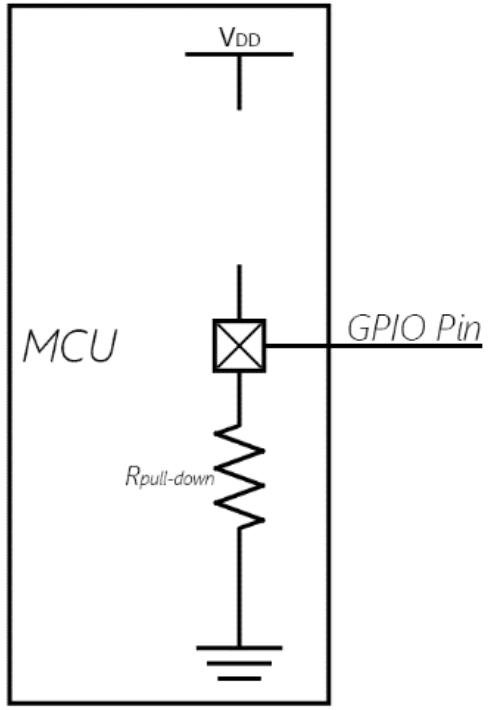
Pull-up

- A **Pull-up input** is a digital input **connected to a power supply** through an **external or internal resistor** called a **pull-up resistor**.
- The pull-up resistor **pulls the input logic level to high** when it is **not driven to low externally**.
- **Pull-up resistor** can be **strong or weak** depending on the value used. Weak pull-up resistor is typically in the range of tens to hundreds of kilo-ohms while strong pull-up resistor is in the range of few kilo-ohms.



Pull-down

- A **Pull-down input** is a digital input **connected to a ground** through an **external or internal resistor** called a **pull-down resistor**.
- The pull-down resistor **pulls the input logic level to low leve**l when it is **not driven to high externally.**
- **Pull-down resistor** can be **strong or weak** depending on the value used. Weak pull-down resistor is typically in the range of tens to hundreds of kilo-ohms while strong pull-down resistor is in the range of few kilo-ohms.

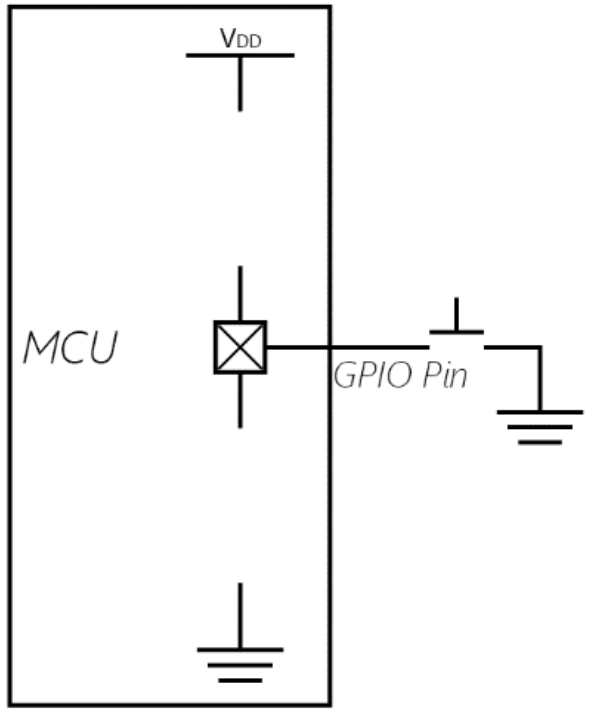


Digital Input

Example

Which digital input type is suitable for the circuit?

Answer: **Pull-up**



It’s  time!

Question 1

Which of the following input types in not recommended as it will cause more power to be wasted as heat?

Answer:

A. Floating

B. Pull-up

C. Pull-down

Question 2

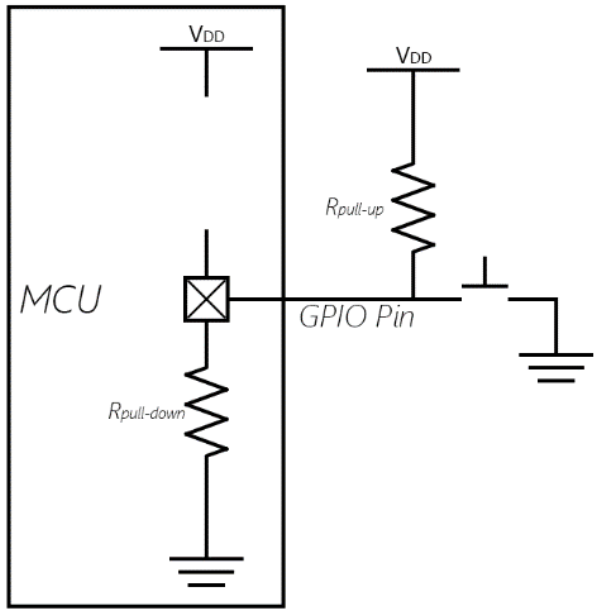
Calculate the R_{pullup} resistance to produce V_{out} of 3 V given $R_{pulldown}$ = 20 kΩ and V_{DD} = 3.3 V.

Answer:

A. 1 kΩ

B. 2 kΩ

C. 3 kΩ



Topics

2.3 Analog Output and Input

- Analog Output
  - Pulse-width Modulation (PWM)
  - Digital-to-analog Converter
- Analog Input
  - Analog Comparator
  - Analog-to-digital Converter

Analog Output

Analog Output

- Analog output in the microcontroller provides a means to **convert digital values** into a **variable voltage** level. This provides an **adjustable output**.

- Microcontrollers are usually **digital based**. However, they often have internal circuitry such as **pulse-width modulation (PWM)** and **digital-to-analog converter (DAC)** which enables them to interface with output analog circuitry.

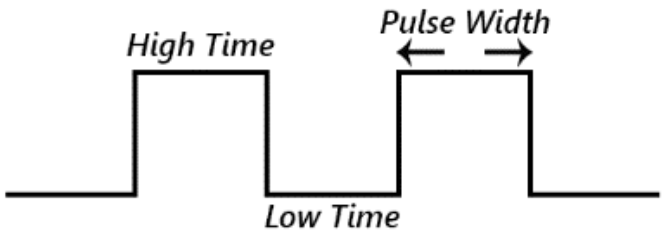
- Analog output is commonly used to control **brightness of RGB LEDs** or to control the **sound of a buzzer**.

Pulse-width Modulation (PWM)

- Pulse Width Modulation (PWM) is a **technique** to **simulate** an **analog voltage** by rapidly switching the digital output between high and low logic.

- This high-low logic pattern can simulate analog voltages in between high and low by **changing the duration** of the time the digital output stays high versus the time it stays low.

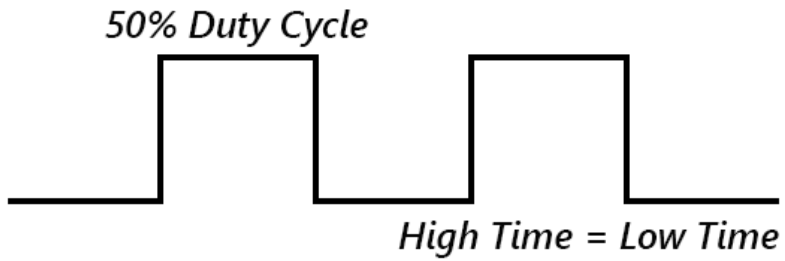
- The **duration** of the time the **digital output stay high** is called the **pulse width**. To get varying analog voltages, change or modulate the pulse width.



PWM Signal

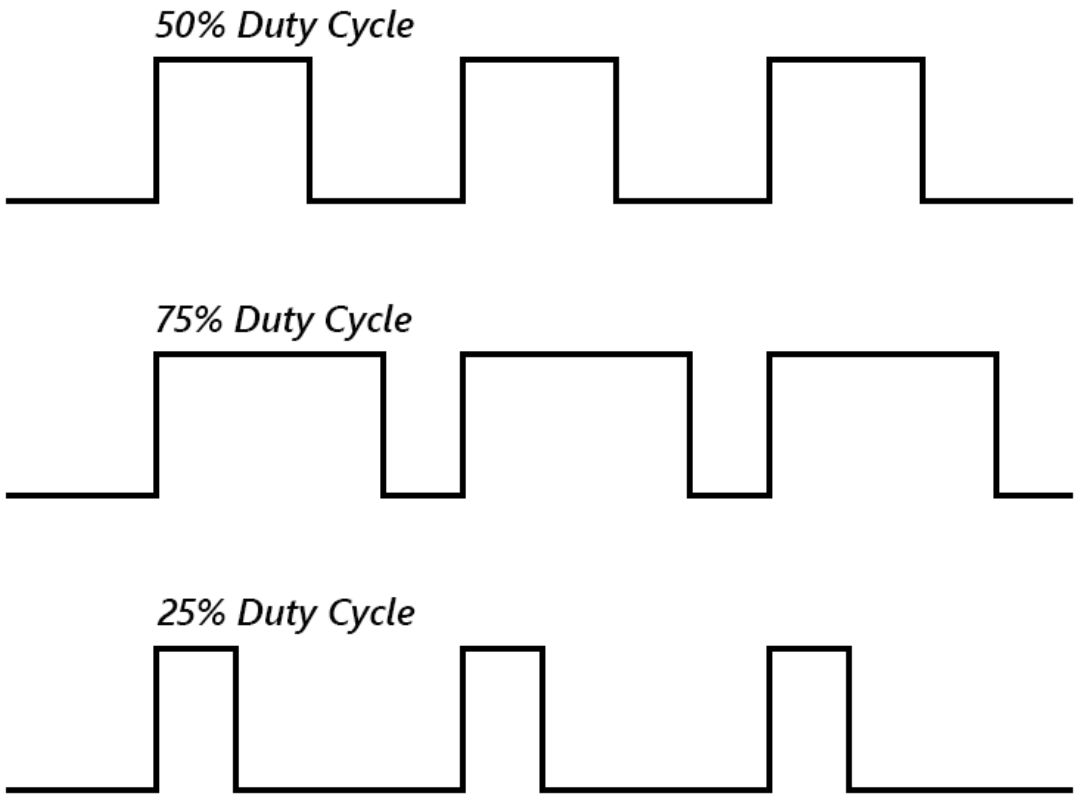
Pulse-width Modulation (PWM)

- To describe the amount of **high time in percentage**, the concept of **duty cycle** is used.
- The **duty cycle** describes the **percentage of time** a digital output **stays high over** an interval or **period of time**.
- If a digital output spends half of the time high and the other half low, the digital output has a **duty cycle of 50%** and resembles an **ideal square wave**.



50% Duty Cycle PWM Signal

Pulse-width Modulation (PWM)

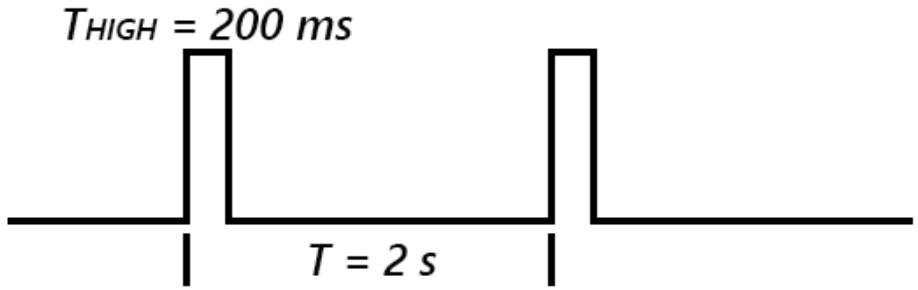


50%, 75%, and 25% Duty Cycle Examples

Pulse-width Modulation (PWM)

Example 1

Calculate the duty cycle of the PWM waveform.



Answer: **Duty Cycle** **= 200ms / 2000ms x 100%**

= 10%

Pulse-width Modulation (PWM)

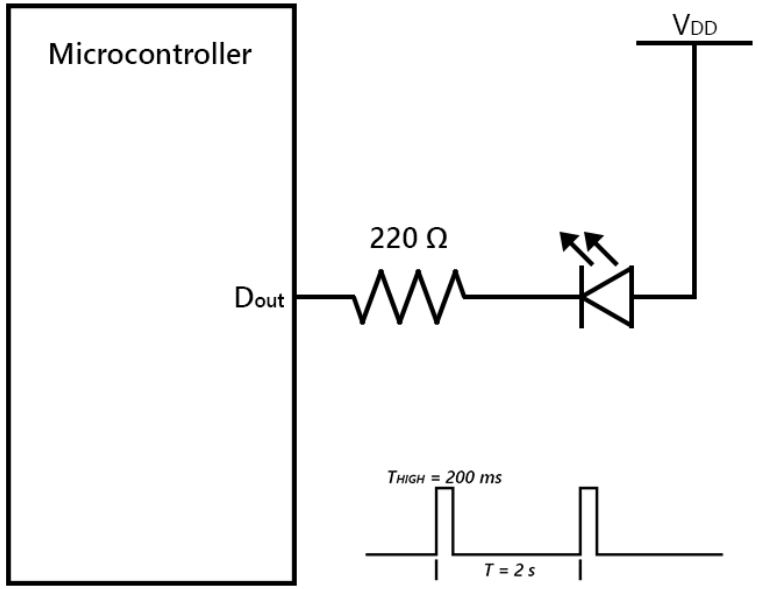
Example 2

Calculate the analog voltage of the D_{out} pin that producing the PWM waveform. Given V_{DD} = 5 V.

Answer: **V_{Dout} = Duty Cycle x 5 V**

= 10% x 5 V

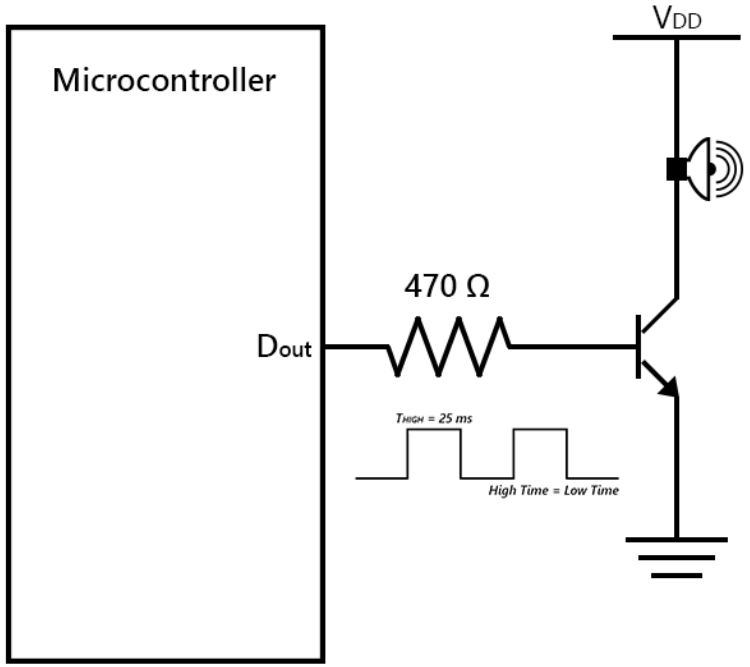
= 0.5 V



Pulse-width Modulation (PWM)

Example 3

What is the frequency of the tone produce at the speaker?



Answer: **$T = T_{HIGH} + T_{LOW}$**

= 25 ms + 25 ms

= 50 ms

F = 1 / T

= 1 / 50 ms

= 20 Hz

Digital-to-analog Converter (DAC)

- Digital-to-Analog Converter (DAC) is an **integrated circuit** that **converts a digital data** such as binary number **to analog signal** such as voltage and current.
- Some of the important characteristics of an DAC are: **Maximum Sampling Rate**, **Resolution** and **Full-scale Voltage.**
- **Maximum Sampling Rate** is the **maximum speed** at which the DACs circuitry can operate and still produce the correct output.
- **Resolution** is the number of **possible output levels** an DAC is designed to reproduce and is usually stated as the **number of bits** it uses.
- **Full-scale voltage** is the maximum analog level an DAC can reach when applying the highest binary code.

Digital-to-analog Converter (DAC)

Example 1

Calculate the total number of non-zero output level of a DAC given the DAC uses **12-bits (n)** for its conversion.

Answer: **Total number of output level** $= 2^n - 1$

$= 2^{12} - 1$

$= 4095$ **levels**

Digital-to-analog Converter (DAC)

Example 2

Calculate the step size of a **10-bits DAC** given its full-scale voltage **V_{FS} = 3.3 V**.

Answer: **Step size** **= V_{FS} / 2^n - 1**

= 3.3 / 2^{10} – 1

= 3.3 / 1023

= 3.23 mV

Digital-to-analog Converter (DAC)

Example 3

Calculate the analog output voltage of a **8-bits DAC** with the digital input of **0x45** given the DAC full-scale voltage **V_{FS} = 5 V**.

Answer:

Step size = V_{FS} / 2^n - 1

= 5 / 2^8 – 1

= 5 / 255

= 19.61 mV

V_{out} = 0x45 x 19.61 mV

= 69 x 19.61 mV

= 1.35 V

It’s  time!

Question 1

Which of the following is NOT the appropriate use of PWM peripheral?

Answer:

A. To adjust the brightness of a light.

B. To produce a tone from a buzzer.

C. To read the analog value of a sensor.

Question 2

Calculate the analog output voltage of a 12-bits DAC with the digital input of 0x132 given the DAC full-scale voltage V_{FS} = 3.3 V.

Answer:

A. 0.247 V

B. 0.106 V

C. 0.806 mV

Analog Input

Analog Input

- Analog input in a microcontroller provides a means to **measure continuous changing conditions** in the physical world such as the changing of environment temperature.

- Microcontrollers are digital based, to **enable the reading** from the analog input, they often have internal circuitry such as **analog comparator** and **analog-to-digital converter (ADC)** to convert the analog value to its digital form.

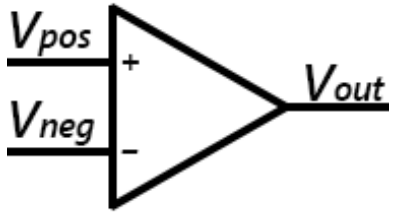
- Analog input is commonly **used to read analog output value** of different sensors such as **temperature or light sensor**.

Analog Input

- Analog Comparator is a device that **compares** the voltage of **two analog inputs** (positive pin and negative pin), with a **digital output** indicating **which input voltage is higher**.

- When the voltage on the **positive pin** is **higher** than the voltage on the **negative pin**, the Analog Comparator outputs a **1**.

- When the voltage on the **negative pin** is **higher** than the voltage on the **positive pin**, the Analog Comparator outputs a **0**.



Analog Comparator

$V_{pos} > V_{neg}$, $V_{out} = 1$

$V_{pos} < V_{neg}$, $V_{out} = 0$

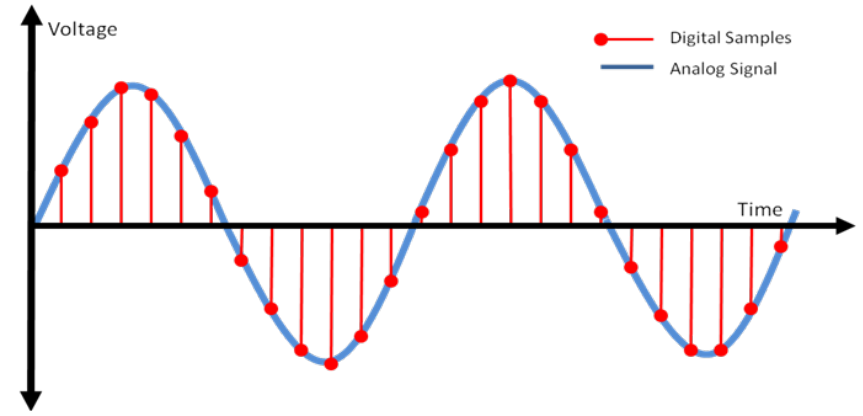
Analog Input

- Analog-to-digital Converter (ADC) is a **integrated circuit** that **converts** a **continuous signal** such as voltage **to a digital number** that represents the **signal's level**.
- The analog to digital conversion involves two processes: **Sampling** and **Quantization**.
- **Sampling** is the process of ~~**reducing**~~ **obtaining the number of points** of a continuous signal.
- **Quantization** is the process of **mapping a large set of analog input values** to a finite set of digital values.

Analog-to-digital Converter (ADC)

- The **speed of the sampling** process requires the calculation of **sampling rate** which refers to **how frequently** an **analog input signal** is converted into a **digital code** and the unit used is **samples per second (SPS)** or **hertz (Hz)**.

- The **faster** the **sampling rate**, the **more accurate** the sampled results will be.

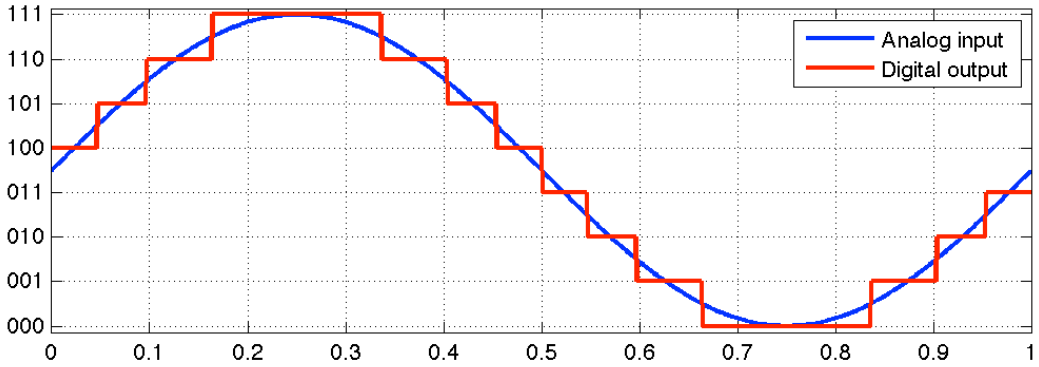


Source: https://bryansiy13.wordpress.com/author/bryansiy/

Analog-to-digital Converter (ADC)

- The **quantization** process requires the **ADC resolution** which is the **number of bits used to represent a sample** to be known.

- The **higher** the **resolution**, the **more accurate** the sampled result will be.

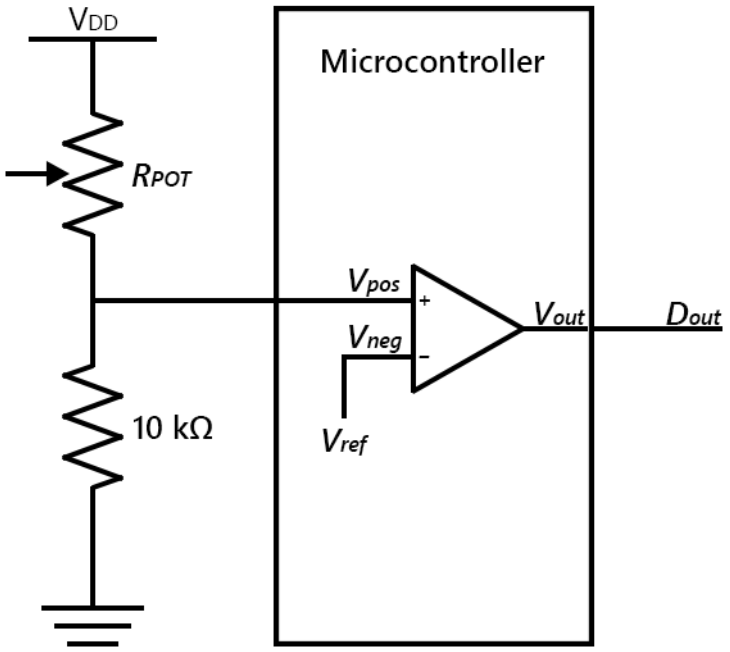


Source: https://dwellangle.wordpress.com/2008/05/21/quantization-in-adcs/

Analog Input

Example

What is the **logic level** of the D_{out} given V_{ref} = 1.6 V, V_{DD} = 3.3 V and R_{POT} = 10 kΩ?



Answer: **V_{pos} = (10 kΩ / 20 kΩ) x 3.3 V**

= 1.65 V

V_{neg} = V_{ref} = 1.6 V

Since V_{pos} > V_{neg}

D_{out} = 1

Analog Input

Example

Calculate the digital output of a **12-bit (n)** ADC given the analog input V_{in} is 2.3 V. The V_{ref} of the ADC is 3.3 V.

Answer: **Digital Output $_{ADC}$** $= (V_{in} / V_{ref}) \times (2^n - 1)$

$= (2.3 / 3.3) \times (2^{12} - 1)$

$= 0.697 \times 4095$

$= 2854$

$= 0x0B26$

Analog Input

Example

Calculate the analog input V_{in} of a **10-bit ADC (n)** given the converted digital output is 0x0284. The V_{ref} of the ADC is 5 V.

Answer: **Digital Output $_{ADC}$**

$$= (V_{in} / V_{ref}) \times (2^n - 1)$$

$$V_{in} = (\text{Digital Output}_{ADC} / (2^n - 1)) \times V_{ref}$$

$$V_{in} = (0x0284 / (2^{10} - 1)) \times 5$$

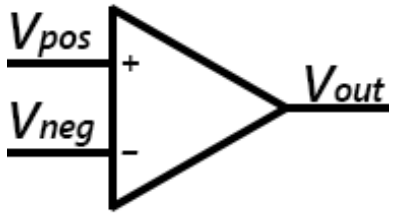
$$V_{in} = (644 / 1023) \times 5$$

$$V_{in} = 3.148V$$

It’s  time!

Question 1

What should be the voltage at Vneg in order to have a logic ‘1’ at Vout of an analog comparator given Vpos = 2.5 V?



Answer:

A. 3.0 V

B. 2.7 V

C. 2.4 V

Question 2

Given an analog signal with maximum frequency of 5 kHz, which sampling rate will give the most accurate sampled results?

Answer:

A. 5 kHz

B. 10 kHz

C. 20 kHz

Question 3

Calculate the analog input V_{in} of a 10-bits ADC (n) given the converted digital output is 0x0284. The V_{ref} of the ADC is 5 V.

Answer:

A. 2.148 V

B. 3.148 V

C. 4.148 V

Topics

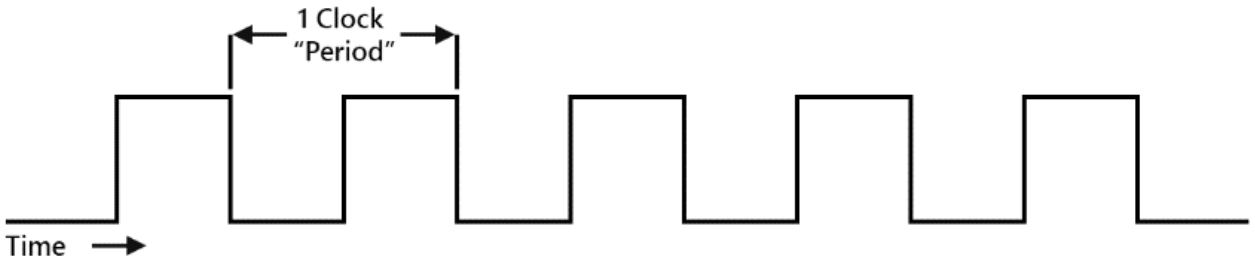
2.4 Clock and Timers

- Clock
  - Oscillator
- Timers
  - Real Time Clock
  - Watchdog Timer

Clock and Timers

Clock

- Clock plays an important roles in a microcontroller. It can be used to:
  - **pace** the **execution of instructions** inside the CPU.
  - **handle** all the **synchronization** and **communications** within the microcontroller.
- Clock is usually generated by an **oscillator** which usually produces **periodic square wave signal** with a **fix frequency**.



Oscillator

- The oscillator used can be **internal** or **external** to a microcontroller.
- The **internal oscillators** are **RC** (resistor, capacitor) **oscillators** where the **external oscillators** are **crystals**, **ceramic resonators** or **crystal oscillator module**.
- The oscillator **frequencies** can range from a **few kHz** up to **several hundred MHz**.
- However, the **maximum oscillator frequency** used always **depends** on the **maximum oscillator frequency supported** by the microcontroller.

Ceramic Resonator

Crystal

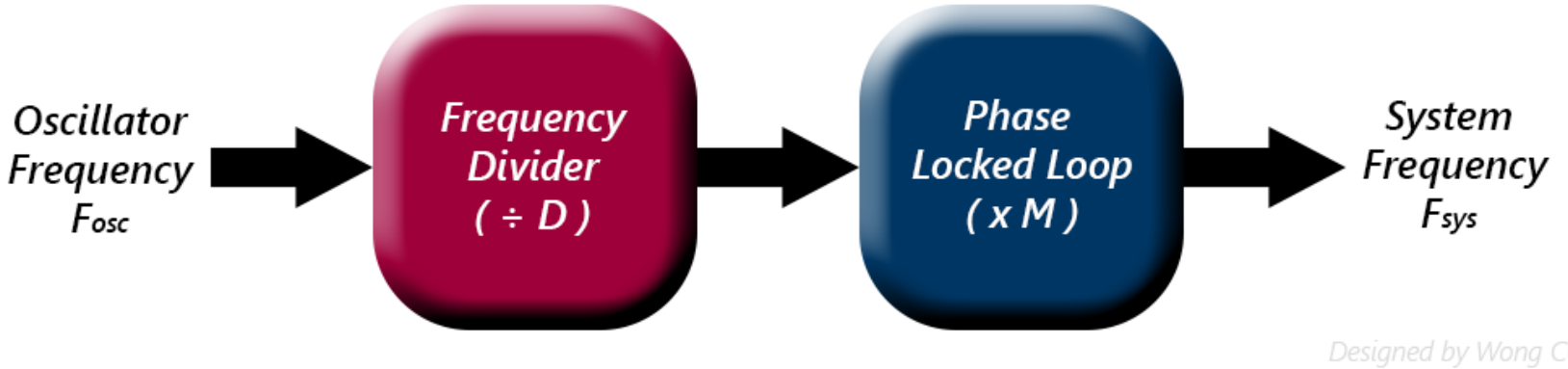
Crystal Oscillator Module

Clock Frequency Derivation Process

- To derive a wide range of frequencies from one oscillator frequency in the microcontroller, **frequency dividers** and **phase locked loop** circuits are often used.
  - **Frequency divider** or **prescaler** is a circuit that **reduces a high frequency signal** to a lower frequency **by an integer division**.
  - **Phase locked loop (PLL)** is a control system that **generates an output signal frequency multiple of the input signal frequency**.

Clock Frequency Derivation Process

- A typical clock frequency derivation process of a microcontroller is shown below.



Clock Frequency Derivation Process

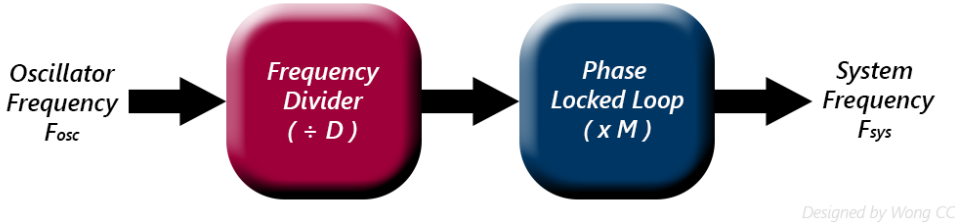
Example:

Calculate the F_{sys} given F_{osc} is 25 MHz, D = 5 and M = 20.

Answer: **F_{sys} $= F_{osc} \div D \times M$**

$= 25\ MHz \div 5 \times 20$

$= 100\ MHz$



Clock Frequency Derivation Process

Example

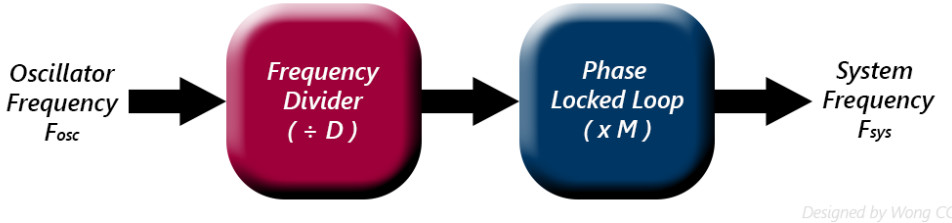
Calculate the F_{osc} given F_{sys} is 80 MHz, D = 2 and M = 8.

Answer: **$F_{sys} = F_{osc} \div D \times M$**

$F_{osc} = F_{sys} \times D \div M$

$= 80\ MHz \times 2 \div 8$

$= 20\ MHz$



Clock Frequency Derivation Process

Example

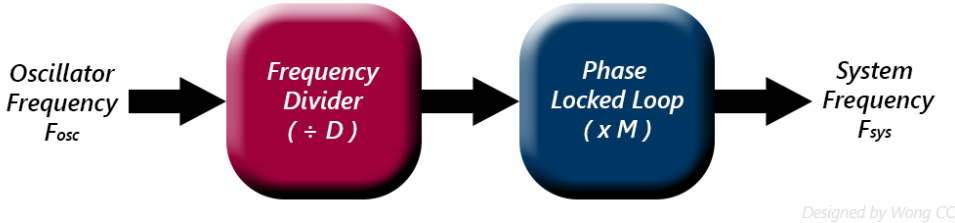
Calculate the Frequency Divider, D given F_{osc} = 4 MHz, M = 90 and F_{sys} is 120 MHz.

Answer: **F_{sys} = F_{osc} ÷ D x M**

D = (F_{osc} x M) ÷ F_{sys}

= (4 MHz x 90) ÷ 120 MHz

= 3



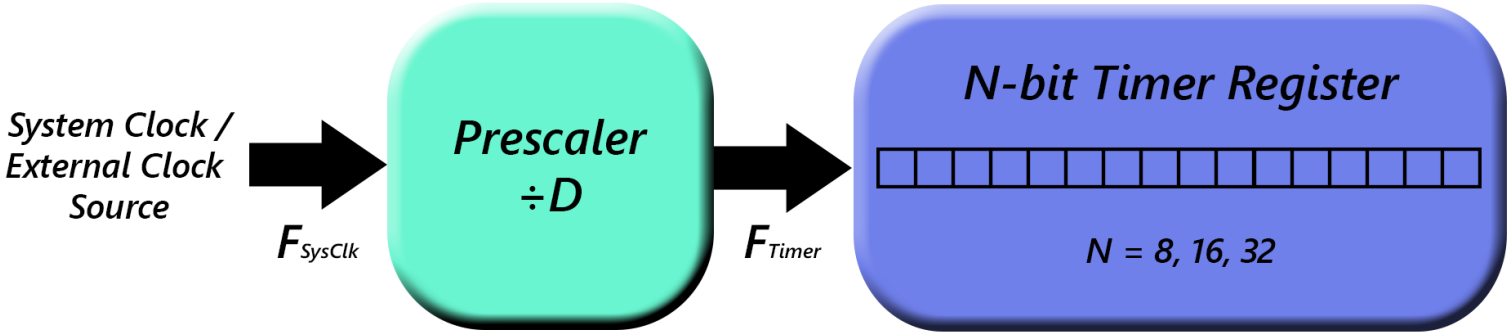
Timers

- Every microcontroller comes with **one** or **more built-in timers**.
- It is usually be configured to **count regular clock pulses to measure elapsed time**.
- It can also be configured to be a **counter to count irregular external event pulses**.
- A typical timer will consist of a **prescaler**, **timer register**, **capture registers**, and **compare registers**.

Timers

- **Prescaler** takes the **basic timer clock frequency** (which may be the CPU system clock frequency) and **divides it by some value** before feeding it to the timer.
- **Timer register** is typically a **n-bit up-counter** (counting from 0 to 2^n-1) or n-bit **down-counter** (counting from 2^n-1 to 0) that responsible for counting.
- **Capture register** is a register which can be **automatically loaded** with the current counter value **upon a transition on an input pin** occurs.
- **Compare register** or match register is used by the Timer register to do comparison. When the value in the **timer register matches** the value in the **compare register** an **event is triggered**.

Timers Generic Structure



Timers Generic Structure

Example

Calculate the time taken for a timer to count down from 0x1234 to 0x1233. Given F_{SysClk} = 100 MHz and D = 5.

Answer: $F_{Timer} = F_{SysClk} / D$

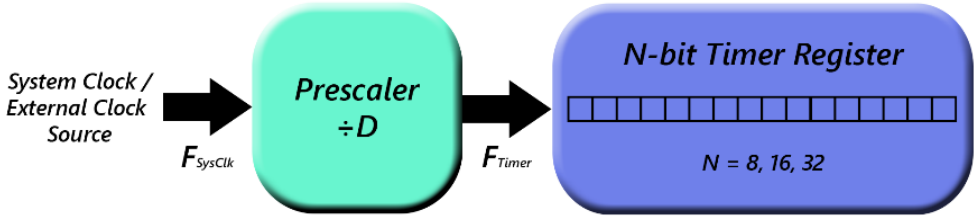
$= 100\ MHz / 5$

$= 20\ MHz$

$T_{1Count} = 1 / F_{Timer}$

$= 1 / 20\ MHz$

$= 50ns$



Timers Generic Structure

Example

Calculate the total time taken for a timer to count down from 0x1234 to 0x0000. Given F_{SysClk} = 120 MHz and D is 10.

Answer: $F_{Timer} = F_{SysClk} / D$
$= 120\ MHz / 10$
$= 12\ MHz$

$C_{Total} = 0x1234 - 0x0000$
$= 0x1234$
$= 4660\ counts$

$T_{1Count} = 1 / F_{Timer}$
$= 1 / 12\ MHz$
$= 83\ ns$

$T_{Total} = 4660 \times 83\ ns$
$= 386{,}780\ ns$
$= 386.78\ us$

Timers Generic Structure

Example

Calculate the total time taken for a timer to count up from 0x2468 to 0xFFFF. Given F_{SysClk} = 80 MHz and D is 8.

Answer: $F_{Timer} = F_{SysClk} / D$
$= 80\ MHz / 8$
$= 10\ MHz$

$C_{Total} = 0xFFFF - 0x2468$
$= 0xDB97$
$= 56215\ counts$

$T_{1Count} = 1 / F_{Timer}$
$= 1 / 10\ MHz$
$= 100\ ns$

$T_{Total} = 56215 \times 100\ ns$
$= 5{,}621{,}500\ ns$
$= 5.622\ ms$

Real Time Clock (RTC)

- Real time clock (RTC) is a timer that is used to **keep track of the current time** upon initialization.
- It is meant to be **low power** consumption and provide **accurate timing**.
- Hence, it often has an **alternate power source** such as battery to **keep the time running** while the primary **power source is off**.
- It is **clocked separately** by a **32.768 kHz** crystal oscillator that produces a **1 Hz internal time reference**.
- Using hardware RTC not only can **free up** the main system for **time-critical tasks**, it can be **more accurate** than software RTC.

Watchdog Timer

- Watchdog Timer is also called **Computer Operating Properly (COP) Timer** is a timer that is used to **detect and recover microcontroller from malfunctions**.

- The Watchdog Timer has the **ability to reset or generate an interrupt** to the microcontroller when the timer elapses.

- To **prevent watchdog timer from elapsing**, **microcontroller will have to regularly restart the watchdog timer** once it is enabled.

- When a microcontroller malfunctions and failed to restart its Watchdog Timer regularly, the microcontroller will be reset or an interrupt will be trigger to diagnose the errors.

It’s  time!

Question 1

Which of the following does not involve in the clock frequency derivation process?

Answer:

A. Amplifier

B. Frequency Divider

C. Phase Lock Loop

Question 2

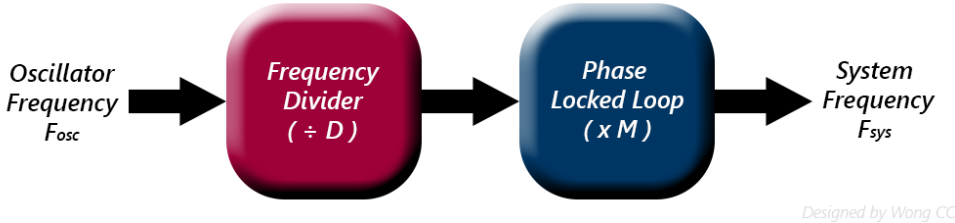
Calculate the Frequency Divider, D given F_{osc} = 4 MHz, M = 90 and F_{sys} is 120 MHz.

Answer:

A. 1

B. 2

C. 3



Question 3

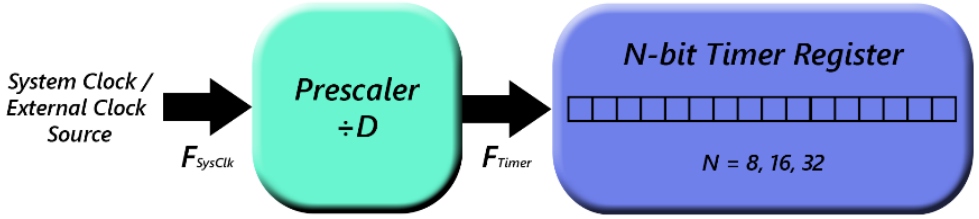
Calculate the total time taken for a timer to count down from 0x1234 to 0x0000. Given F_{SysClk} = 120 MHz and D is 12.

Answer:

A. 466 us

B. 646 us

C. 664 us



Topics

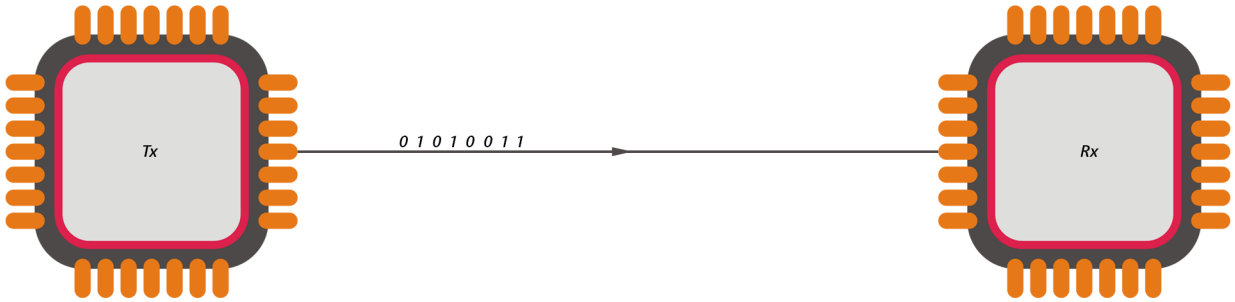
2.5 Serial Communications Interfaces

- Inter-Integrated Circuit (I^2C)
- Universal Asynchronous Receiver / Transmitter (UART)

Serial Communications Interfaces

Serial Communications Interfaces

- **Serial** communications is the **process** of **sending one data bit** at a time **sequentially** over a single wire.



Synchronous vs Asynchronous

- Serial communications can be further categorised into:
  - **Synchronous**
  - **Asynchronous**

- **Synchronous** serial communications established the communication by using a **common clock signal** connecting both endpoints. The common communication protocol using synchronous serial communications is **Inter-Integrated Circuit (I^2C)**.

- **Asynchronous** serial communications established the communication **without using a common clock signal** connecting both endpoints. The common communication protocol using asynchronous serial communications is **Universal Asynchronous Receiver / Transmitter (UART)**.

Inter-Integrated Circuit (I^2C)

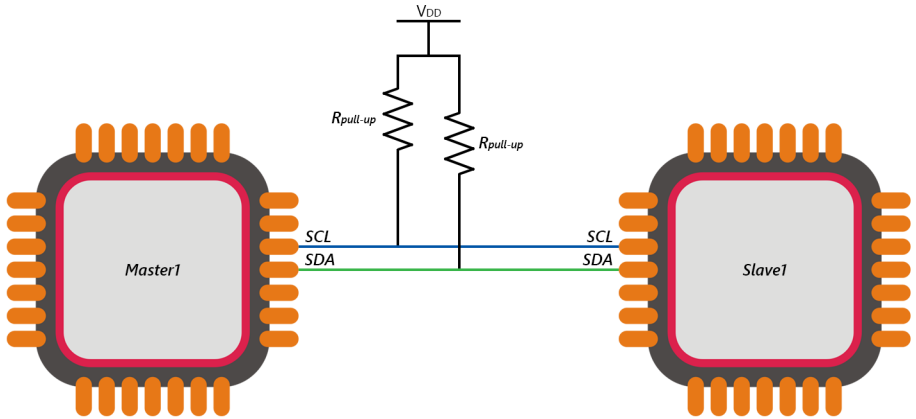
- **Inter-Integrated Circuit (I^2C)** is a **synchronous** serial communications protocol developed by **NXP** Semiconductors to be used for **short distance** communication.

- It is very often used in communicating with **EEPROMs** and different type of **sensors**.

- I^2C communicates using a **master-slave communications model** with **multiple masters** and **multiple slaves** where multiple masters can be connected to a single slave or multiple slaves and vice versa.

Inter-Integrated Circuit (I^2C)

- I^2C uses only **two** signal lines namely SCL and SDA.



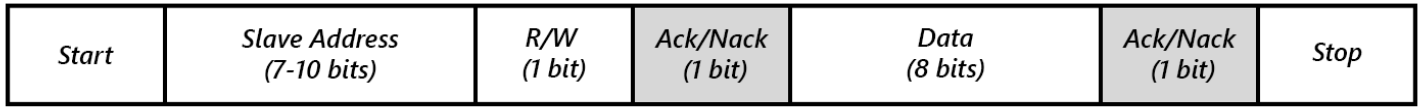
I^2C Connections

Inter-Integrated Circuit (I^2C)

- SCL – Serial Clock
  - Generated by **master device** using frequency supported by the slave device. Typical supported frequencies are **100 Kbps**, **400 Kbps**, **3.4 MHz** and **5 MHz**.
  - Is **open-drained** and connected to a power supply via a pull-up resistor.
- SDA – Serial Data
  - Used by **master and slave** device to **send and receive data**.
  - Is **open-drained** and connected to a power supply via a pull-up resistor.

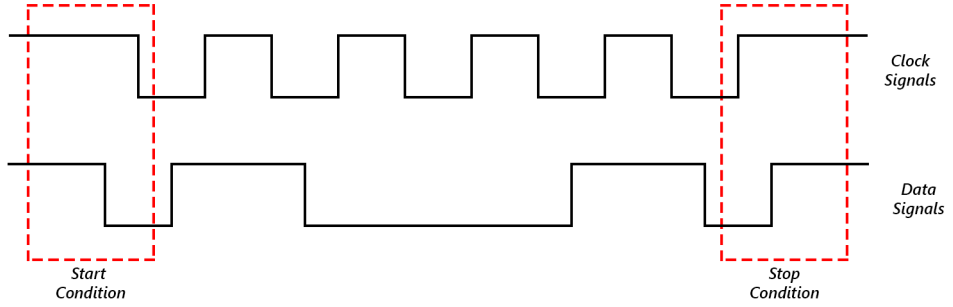
Inter-Integrated Circuit (I^2C)

- I^2C Data Packet
  - I^2C transfers data in packets. Each packet consists of **start** and **stop conditions**, **read/write bits**, **ACK/NACK bits**, **slave address frame** and **one or more data frames**.



Inter-Integrated Circuit (I^2C)

- Start Condition
  - The **SDA line** switches **from high to low** logic level **before** the **SCL line**.
- Stop Condition
  - The **SDA line** switches from **low to high** logic level **after** the **SCL line**.



Inter-Integrated Circuit (I^2C)

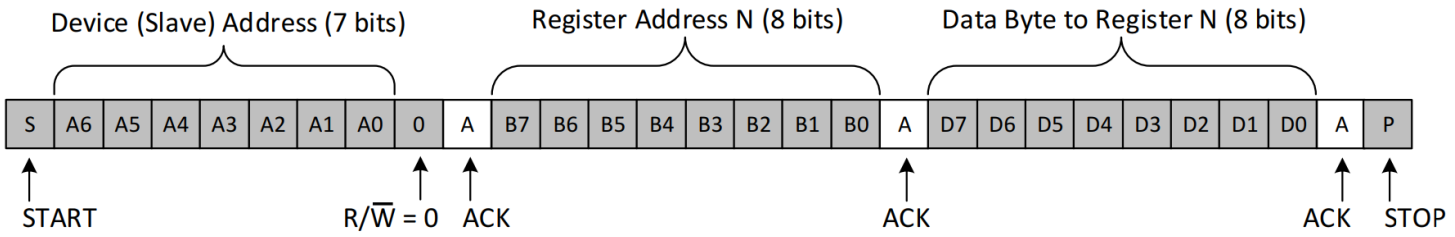
- Slave Address Frame
  - A 7 or 10 bit sequence unique to **identify the slave device**.
- Read/Write Bits
  - A **single bit** specifying whether the master device is **sending data** to the slave device (write, **0**) or **requesting data** from it (read, **1**).

Inter-Integrated Circuit (I^2C)

- ACK/NACK Bit
  - Each frame in a packet is followed by an acknowledge/no-acknowledge bit. If an address frame or **data frame was successfully received by slave device**, an **ACK bit is returned** to the master device.
- Data Frame
  - A **8 bits frame** allocated for data transfer.

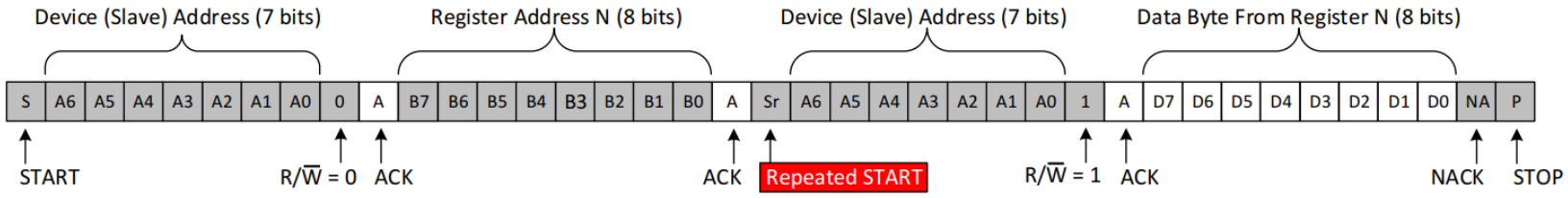
Inter-Integrated Circuit (I^2C)

- I^2C Data Transmission Method – Master Write
  - Master-transmitter **sends a START condition** and **addresses** to the slave-receiver.
  - Master-transmitter **sends data to slave-receiver.**
  - Master-transmitter **terminates the transfer** with a **STOP condition.**



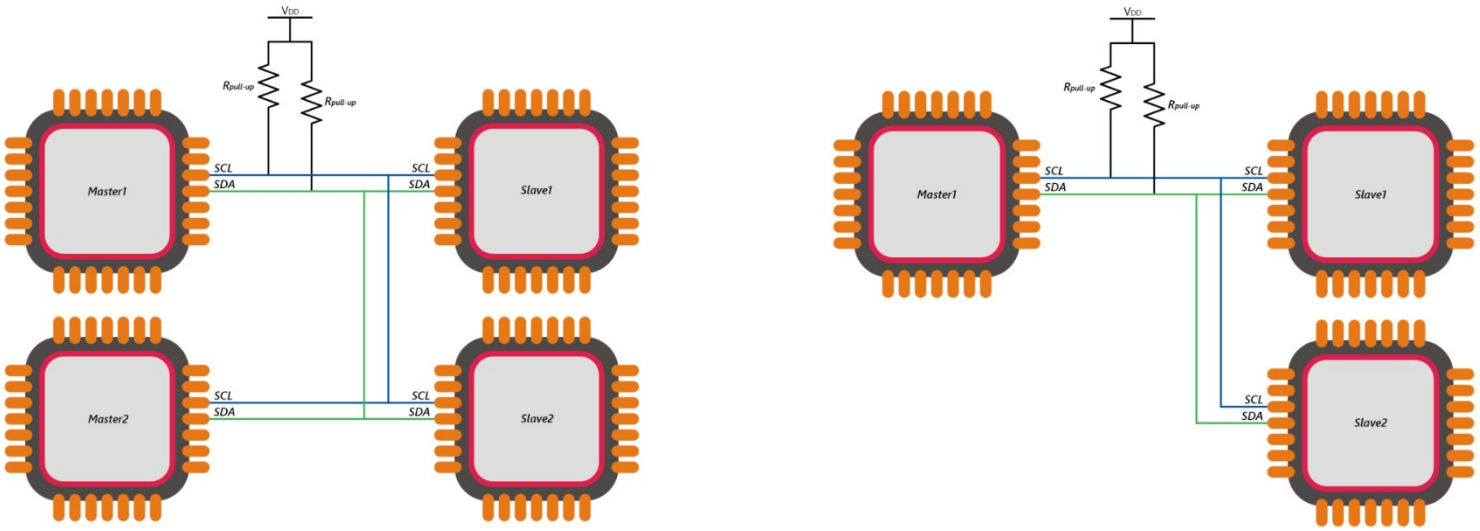
Inter-Integrated Circuit (I^2C)

- I^2C Data Transmission Method – Master Read
  - Master-receiver **sends a START condition** and **addresses** the slave-transmitter.
  - Master-receiver **sends the requested register to read** to slave-transmitter.
  - Master-receiver **receives data from the slave-transmitter**.
  - Master-receiver **terminates the transfer** with a **STOP condition.**



Inter-Integrated Circuit (I^2C)

- I^2C connections configuration with **multiple master and slaves.**



I^2C Multiple Master and Slave Configuration

Universal Asynchronous Receiver / Transmitter

- Universal Asynchronous Receiver / Transmitter (UART) is an **asynchronous** serial communications protocol that used to **transmit** and **receive** data **peer-to-peer**.

- The term asynchronous means both the **transmitter** and **receiver** use their **own clock source** to generate clock timing. However, the receiver must know the incoming clock timing and have a **matching clock timing** in order to communicate with each other.

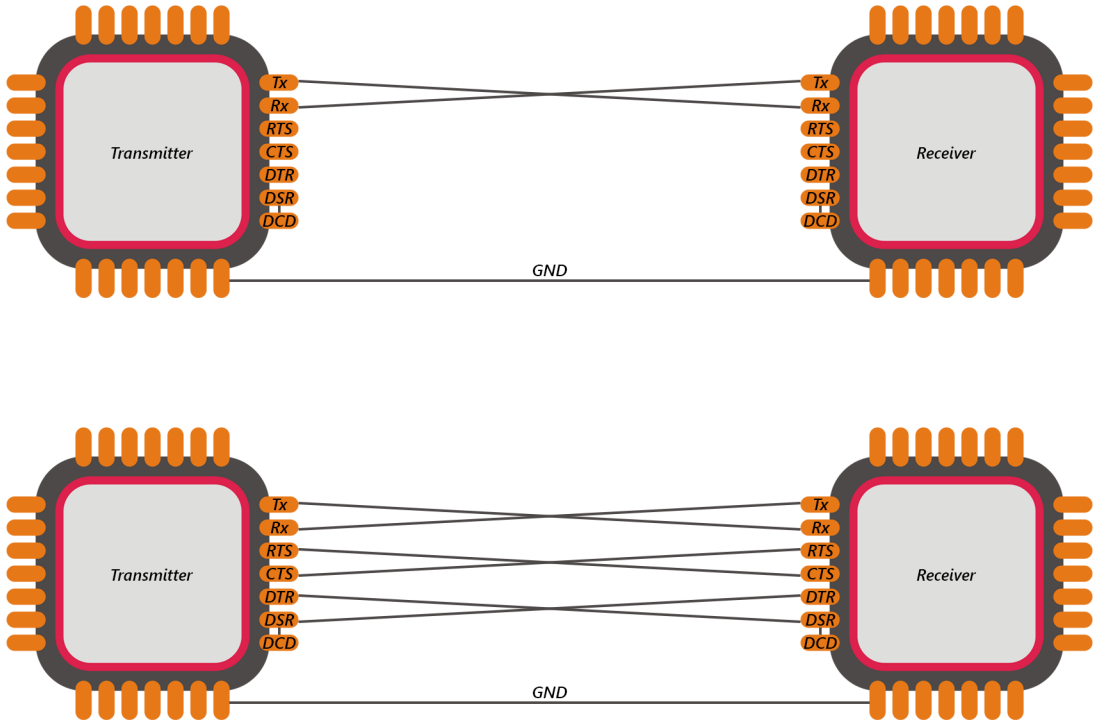
- The **matching clock timing** is also known as **Baud Rate** which is the **rate in bps** at which information is transferred in a communication channel.

Universal Asynchronous Receiver / Transmitter

- UART uses **two signal lines**, **Tx** and **Rx** for data transmission **without hardware flow control**.

- When **hardware flow control (Full modem control)** is used, additional **6 signal lines is used** namely RTS (Request To Send), CTS (Clear To Send), DTR (Data Terminal Ready), DSR (Data Set Ready), DCD (Data Carrier Detect) and RI (Ring Indicator).

Universal Asynchronous Receiver / Transmitter



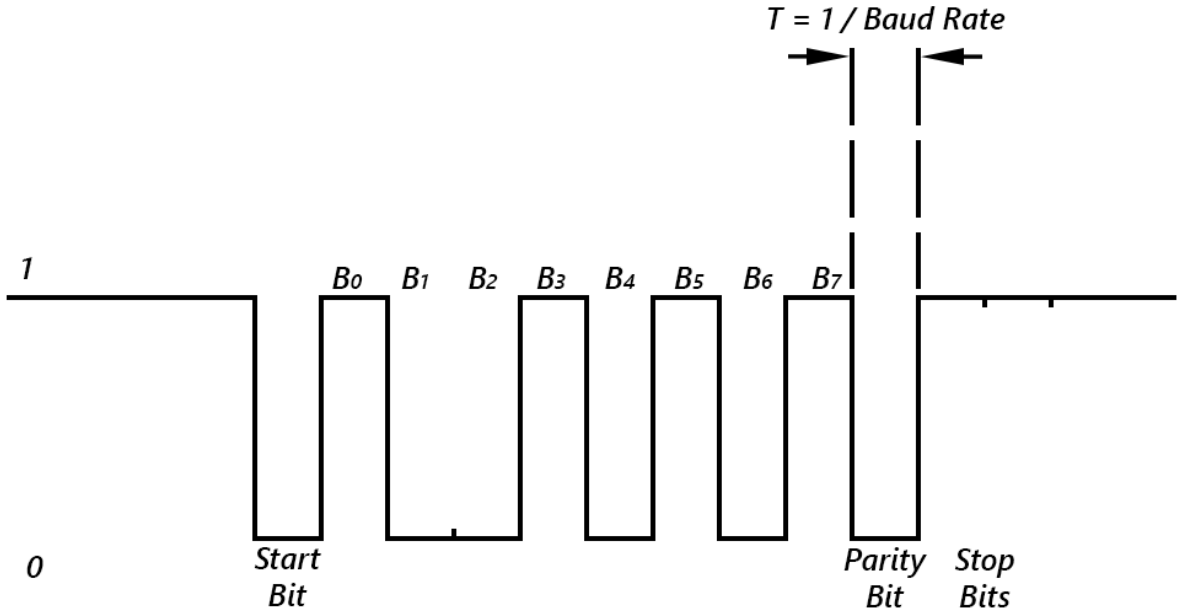
UART communications setup with and without hardware flow control

Universal Asynchronous Receiver / Transmitter

- Tx – Transmit line
  - Often called data out(DOUT).
  - Used by **transmitter** to **send data** to the **receiver**.
- Rx - Receive line
  - Often called data in(DIN).
  - Used by **receiver** to **receive** data from the transmitter.

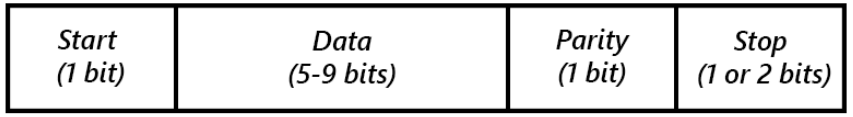
Universal Asynchronous Receiver / Transmitter

- UART Waveform



Universal Asynchronous Receiver / Transmitter

- The UART transmitted data is organized into packets with **1 start bit**, **5 to 9 data bits**, an optional **parity bit**, and **1 or 2 stop bits**.



Universal Asynchronous Receiver / Transmitter

- Start Bit
  - The UART data transmission line is normally **held at a logic level high** when it is **idle**. To **start the transfer** of data, the **transmitter pulls** the transmission line from **high to low**. The receiver begins reading the bits in the data frame when it detects the high to low level transition.
- Data Frame
  - The data frame can contains **5 to 8 data bits long** if a parity bit is used, else the data frame can be **9 bits long**. Data bits is sent with the **least significant bit (LSB)** first.

Universal Asynchronous Receiver / Transmitter

- Parity Bit
  - The parity bit is a way for the receiving UART to **check if any data has changed** during the transmission.
  - When **even parity** is used, the '**1**' bits in the data frame and the attached parity bit should total to an **even number** to indicate **error free** in the data frame.
  - When **odd parity** is used, the '**1**' bits in the data frame and the attached parity bit should total to an **odd number** to indicate **error free** in the data frame.
- Stop Bit
  - The **transmitter drives** the data transmission line from a **low to high** for **at least two bit durations** to signal the end of the data packet to the receiver.

Universal Asynchronous Receiver / Transmitter

- In order for both UART transmitter and receiver to communicate, the following settings needs to be setup at both side: **Baud Rate, Data Bits, Parity and Stop Bits**.
  - Baud Rate: 200bps, 4800bps, 9600bps, 19200bps, 38400bps and 115200bps
  - Data Bits: 5 – 9 bits
  - Parity: Even or Odd
  - Stop Bits: 1 or 2 bits

It’s  time!

Question 1

Which of the following frame or bit does not belong to the I^2C data packet?

Answer:

A. Slave address frame

B. R/W bit

C. Parity bit

D. Ack/Nack bit

Question 2

What is needed to be done by the UART transmitter to start the data transfer?

Answer: A. The transmitter pulls the transmit line from low to high.

B. The transmitter pulls the transmit line from high to low.

End of Chapter 2